

Modulo I

Introduzione

- Terminologia
- STRIDE
- Triade CIA
- Il protocollo HTTP e minacce correlate
- Browser Security Model

TERMINOLOGIA

Sicurezza = Protezione contro le minacce

ATTACCO

Tentativo di abuso di un sistema

MINACCIA

Un potenziale attacco

VULNERABILITA'

Una debolezza che può essere abusata

DEBOLEZZA

Un costrutto errato che può ridurre la sicurezza

TERMINOLOGIA

Sicurezza = Protezione contro le minacce

EXPLOIT

Metodo per abusare una vulnerabilità specifica

RISCHIO

Probabilità di perdita di valore a causa di attacchi riusciti

MITIGAZIONE

Misura per diminuire l'impatto di una minaccia

PATCH

Misura che rimuove una vulnerabilità da un sistema

Spoofing

Tampering

Repudiation

Information Disclosure

Denial of service

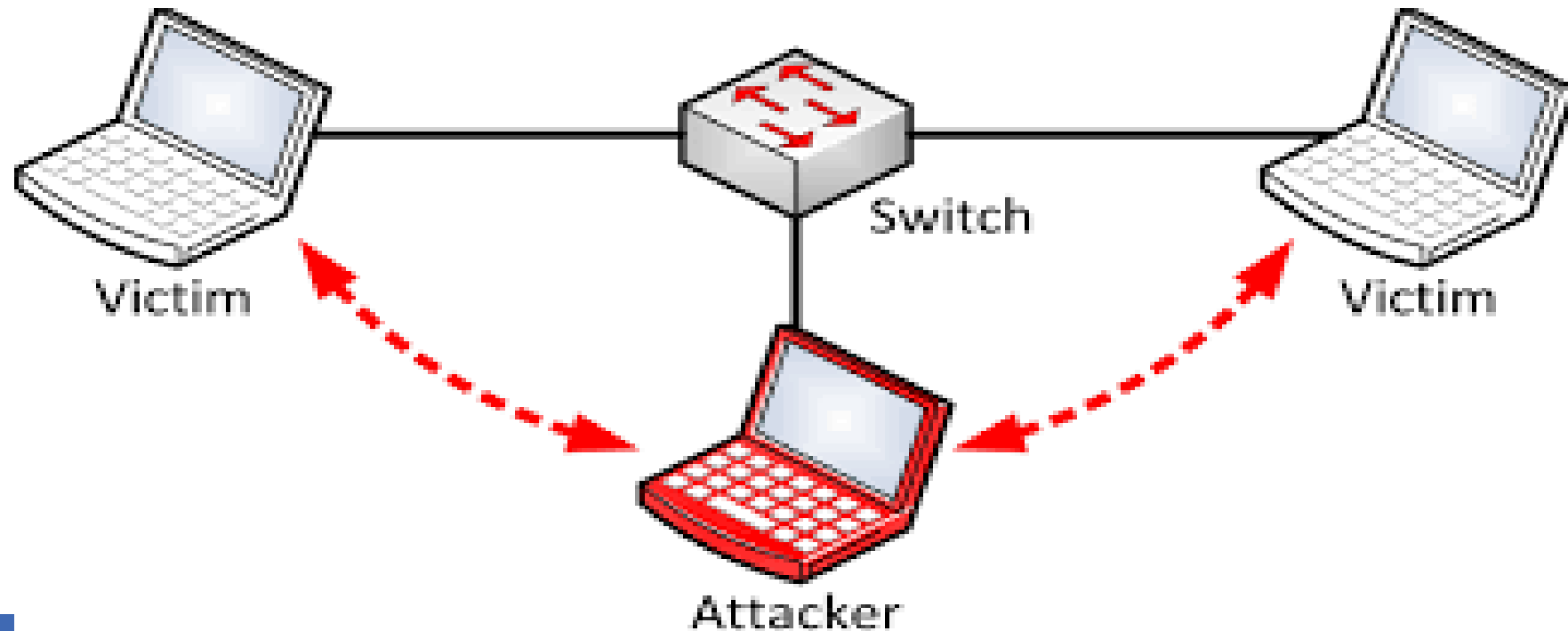
Elevation of privilege

Termine forgiato nel 2006
da Hernan, Lambert,
Hoswald & Shostack

Descrive le classi di
tipologia di attacco

STRIDE - SPOOFING

Pretendere di essere qualcosa o qualcuno diverso da ciò che si è in realtà. Comprende tutte le problematiche relative a identificazione e autenticazione



Modifica di dati memorizzati o in transito



STRIDE - REPUDIATION

Negare di aver fatto o di non aver fatto qualcosa



STRIDE - INFORMATION DISCLOSURE

Accedere ad informazioni che non dovrebbero essere viste

```
Microsoft OLE DB Provider for ODBC Drivers error '80040e14'  
[Microsoft][ODBC SQL Server Driver][SQL Server]Incorrect syntax near the keyword 'or'.  
/login.asp, line 11
```

Server Error in '/' Application.

A network-related or instance-specific error occurred while establishing a connection to SQL Server. The server was not found or was not accessible. Verify that the instance name is correct and that SQL Server is configured to allow remote connections. (provider: Named Pipes Provider, error: 40 - Could not open a connection to SQL Server)

Description: An unhandled exception occurred during the execution of the current web request. Please review the stack trace for more information about the error and where it originated in the code.

Exception Details: System.Data.SqlClient.SqlException: A network-related or instance-specific error occurred while establishing a connection to SQL Server. The server was not found or was not accessible. Verify that the instance name is correct and that SQL Server is configured to allow remote connections. (provider: Named Pipes Provider, error: 40 - Could not open a connection to SQL Server)

Source Error:

```
Line 15:      string conString = @"Data Source=192.168.10.120;Initial Catalog=Northwind;Integrated Security=SSPI";  
Line 16:      SqlConnection con = new SqlConnection(conString);  
Line 17:      con.Open();  
Line 18:      string qry = "select UName,UPass from UserDetails";  
Line 19:
```

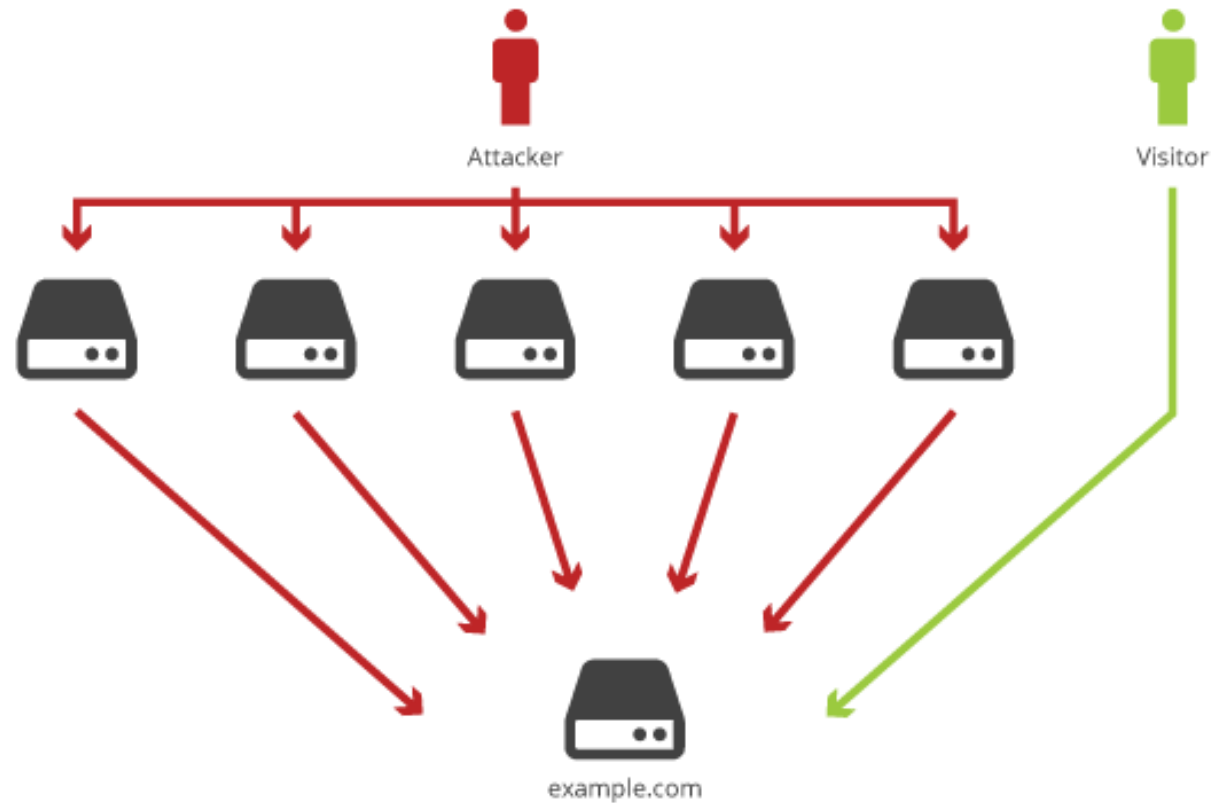
Source File: D:\utility\WebApplication1\WebApplication1\Login.aspx.cs Line: 17

▼ Response Headers [view source](#)

```
Cache-Control: private  
Content-Encoding: gzip  
Content-Length: 1208  
Content-Type: text/html; charset=utf-8  
Date: Wed, 18 Feb 2015 02:56:15 GMT  
Server: Microsoft-IIS/8.0  
Vary: Accept-Encoding  
X-AspNet-Version: 4.0.30319  
X-AspNetMvc-Version: 3.0  
X-Powered-By: ASP.NET
```

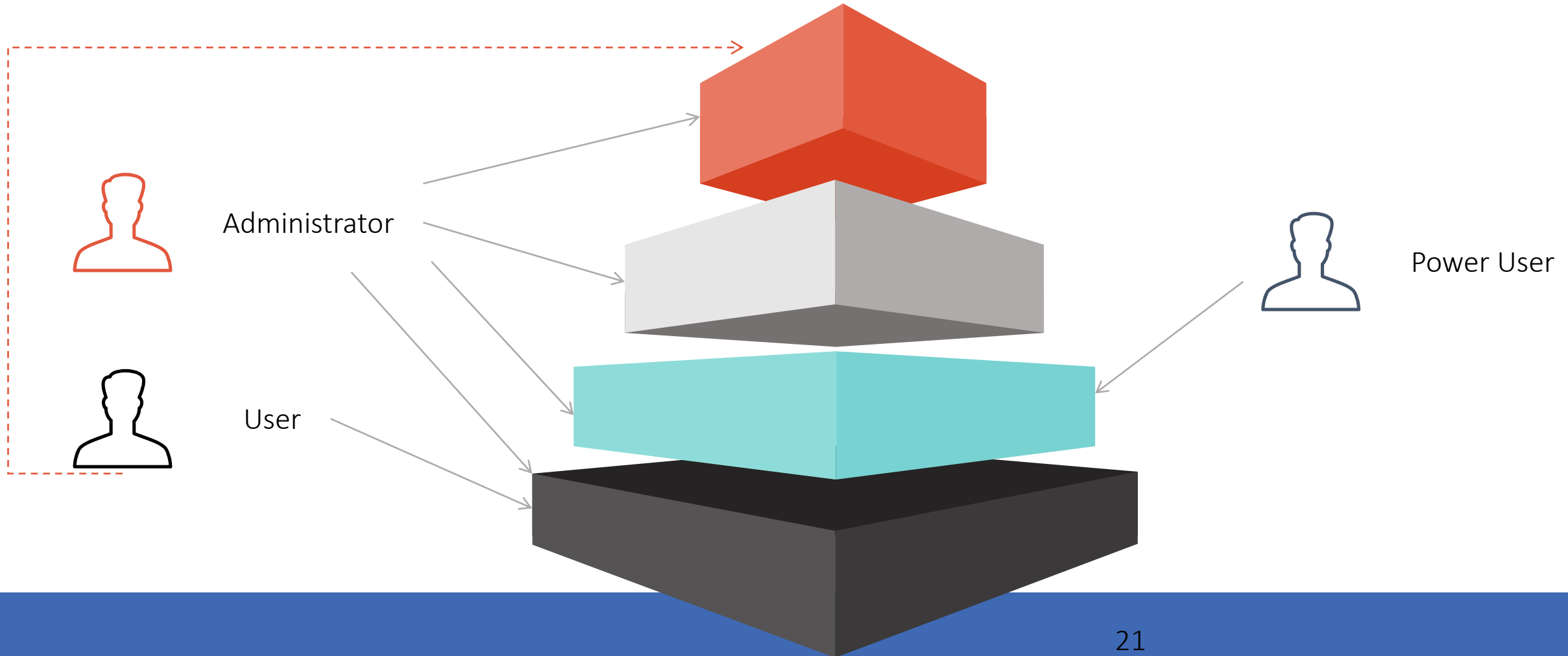

STRIDE - DENIAL OF SERVICE

Rendere un sistema non disponibile



STRIDE - ELEVATION OF PRIVILEGE

Fare qualcosa che non è permesso



ATTACK ZONES - ATTACK SURFACES

ATTACK SURFACE

PARTI DI UN SISTEMA A CUI UN
ATTACKER PUO' AVERE ACCESSO

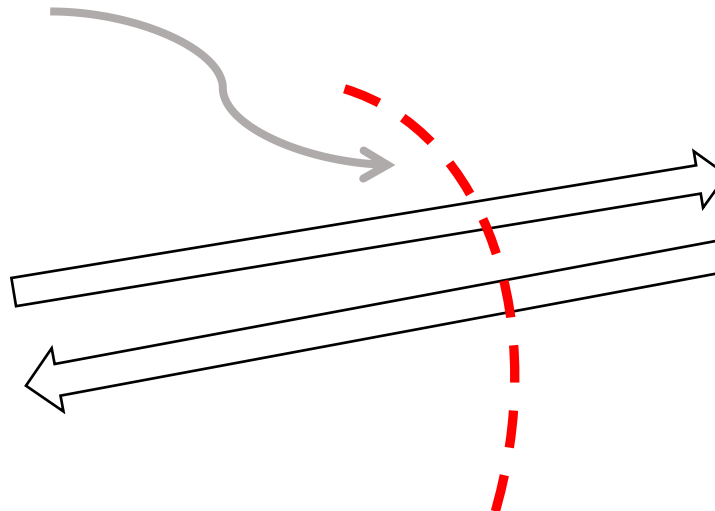
TRUST ZONES

AREE DEL SISTEMA CHE
POSSIAMO RITENERE AFFIDABILI

TRUST BOUNDARIES

CONDIFINI TRA TRUST ZONES
CANDIDATI IDEALI PER ESSERE ATTACK
SURFACES

L' ESPOSIZIONE ALLA RETE DEL SERVER E'
UN TRUST BOUNDARY



UN WEB SERVER E'
UNA TRUST ZONE

Modello utilizzato per la realizzazione di browsers sicuri, dovuto principalmente all'utilizzo di contenuto dinamico

Same Origin Policy (SOP)

Descrive sotto quali condizioni scripts e programmi nel contesto di un applicazione web possono accedere al contenuto di un'altra applicazione web.

In particolare previene che scripts caricati da un sito possano ottenere o impostare proprietà di un documento caricato da un sito diverso, o che richieste possano arrivare da origini diverse da quella del sito in questione

WEB SECURITY - SAME ORIGIN POLICY

ORIGINE: combinazione di host name, protocollo, e numero di porta.
Same origin: tutte le richieste consentite arrivano dalla stessa **origine**

Esempio: <http://www.mydomain.com> (default port 80)

URL	ESITO	MOTIVO
http://www.mydomain.com /index.html	OK	Stessa origine
http://www.mydomain.com /dir/page1.html	OK	Stessa origine
http://www.mydomain.com :8080/dir/page1.html	FAIL	Numero di porta diverso
https://www.mydomain.com /dir/page1.html	FAIL	Protocollo diverso
https://it.mydomain.com /dir/page1.html	FAIL	Host name diverso

Nelle moderne applicazioni web è nata la necessità di eseguire chiamate a servizi esposti (come le API) ospitate da domini diversi e che il meccanismo SOP non ci consente di fare.

Come fare?

Cross Origin Resource Sharing

Permette al browser di effettuare chiamate cross-origin in base alla presenza ed il valore dell'header HTTP Access-Control-Allow-Origin

Access-Control-Allow-Origin: * permette chiamate Cross-Origin

Access-Control-Allow-Origin: xxx.xxx.xxx.xxx permette chiamate cross-origin dall'url/IP specificato

Access-Control-With-Credential (optional): true permette l'invio dei cookie di autenticazione, disabilitato di default per le chiamate CORS



<https://www.owasp.org>

OWASP (Open Web Application Security Project) è una fondazione senza scopo di lucro fondata nel 2001.

Tutto il materiale messo a disposizione sul sito dell'organizzazione è free e Open Source.

Scopo di OWASP è fornire un po' di conoscenza per quanto riguarda la sicurezza, a partire dalle web applications, alle app per dispositivi mobile, IoT.

Esistono vari progetti attivi, tutti consultabili dal sito dell'organizzazione, in ambito di sviluppo, modellazione, test di applicazioi web.

OWASP TOP 10 PROJECT

Uno dei progetti più importanti dell' organizzazione OWASP in cui sono presenti le 10 minacce più importanti per la sicurezza delle applicazioni web, raggiungibile al link https://www.owasp.org/index.php/Top_10_2013

OWASP TOP 10 - 2013

A1 - INJECTION

A2 – AUTHENTICATION WEAKNESS
AND SESSION MANAGEMENT

A3 – XSS CROSS SITE SCRIPTING

A4 - INSECURE DIRECT OBJECT
REFERENCES

A5 – SECURITY MISCONFIGURATION

A6 – SENSITIVE DATA EXPOSURE

A7 – MISSING FUNCTION LEVEL
ACCESS CONTROL

A8 –CROSS SITE REQUEST FORGERY
(CSRF – XSRF)

A9 – USING COMPONENTS WITH
KNOWN VULNERABILITY

A10 – UNVALIDATED REDIRECTS AND
FORWARDS

OWASP TOP 10 PROJECT

OWASP TOP 10 2013	OWASP TOP 10 2017
A1:2013 - Injection	A1:2017 – Injection
A2:2013 - Broken Authentication an Session Management	A2:2017 - Broken Authentication
A3:2013 - Cross-Site Scription (XSS)	A3:2017 - Sensitive Data Exposure
A4:2013 - Insecure Direct Object References	A4:2017 – XML External Entities (XXE)
A5:2013 - Security Misconfiguration	A5:2017 – Broken Access Control
A6:2013 - Sensitive Data Exposure	A6:2017 – Security Misconfiguration
A7:2013 - Missing Function Level Access Control	A7:2017 - Cross-Site Scription (XSS)
A8:2013 - Cross Site Request Forgery (CSRF)	A8:2017 – Insecure Deserialization
A9:2013 - Using Component With Known Vulnerabilities	A9:2017 - Using Component With Known Vulnerabilities
A10:2013 - Unvalidated Redirect And Forwards	A10:2017 – Insufficient Logging & Monitoring

OWASP TOP 10 PROJECT

OWASP TOP 10 2017	OWASP TOP 10 2021
A1:2017 – Injection	A1:2021 – Broken Access Control
A2:2017 - Broken Authentication	A2:2021 – Cryptographic Failures
A3:2017 - Sensitive Data Exposure	A3:2021 - Injection
A4:2017 – XML External Entities (XXE)	A4:2021 - Insecure Design
A5:2017 – Broken Access Control	A5:2021 - Security Misconfiguration
A6:2017 – Security Misconfiguration	A6:2021 – Vulnerable and Outdated Components
A7:2017 - Cross-Site Scripting (XSS)	A7:2021 – Identification and Authentication Failures
A8:2017 – Insecure Deserialization	A8:2021 – Software and Data Integrity Failures
A9:2017 - Using Component With Known Vulnerabilities	A09:2021-Security Logging and Monitoring Failures
A10:2017 – Insufficient Logging & Monitoring	A10:2021-Server-Side Request Forgery

Il primo progetto Top 10 è nato per la sicurezza delle applicazioni web. Con l'evolvere della tecnologia e del panorama delle minacce e vulnerabilità, sono state sviluppate anche altre Top 10 specifiche per tecnologia, e altre sono in fase di sviluppo (ad esempio Desktop App).

- OWASP Top Ten (applicazioni web) <https://owasp.org/Top10/>
- Mobile Top Ten <https://owasp.org/www-project-mobile-top-10/>
- Top Ten Client-Side Security Risk <https://owasp.org/www-project-top-10-client-side-security-risks/>
- Top Ten Privacy Risk <https://owasp.org/www-project-top-10-privacy-risks/>
- Top Ten API Security <https://owasp.org/www-project-api-security/>
- Top Ten Serverless <https://owasp.org/www-project-serverless-top-10/>
- Top Ten Docker <https://owasp.org/www-project-docker-top-10/>

Modulo II

Autenticazione e gestione della sessione

- Problemi legati all'utilizzo delle password
- Principi di gestione delle password
- Gestione della sessione
- Problemi relative alla gestione della sessione
- Best practices relative alla gestione della sessione
- Cross Site Request Forgery e Clickjacking

Misura per determinare la controparte con cui comunichiamo

Metodologie di autenticazione

- Qualcosa che sappiamo: es. passwords
- Qualcosa di cui abbiamo bisogno: es. Smart card
- Qualcosa che ci appartiene: es. Misure biometriche
- Qualcosa relativo al contesto : es. Device, posizione, rete di accesso

Autenticazione multi-fattore (MFA): combinazione di più metodologie di autenticazione

Es. Autenticazione a due fattori: password + impronta digitale

Segreto condiviso tra due parti

Proteggere password in transito

HTTPS

Proteggere password memorizzate

SALTING + HASHING

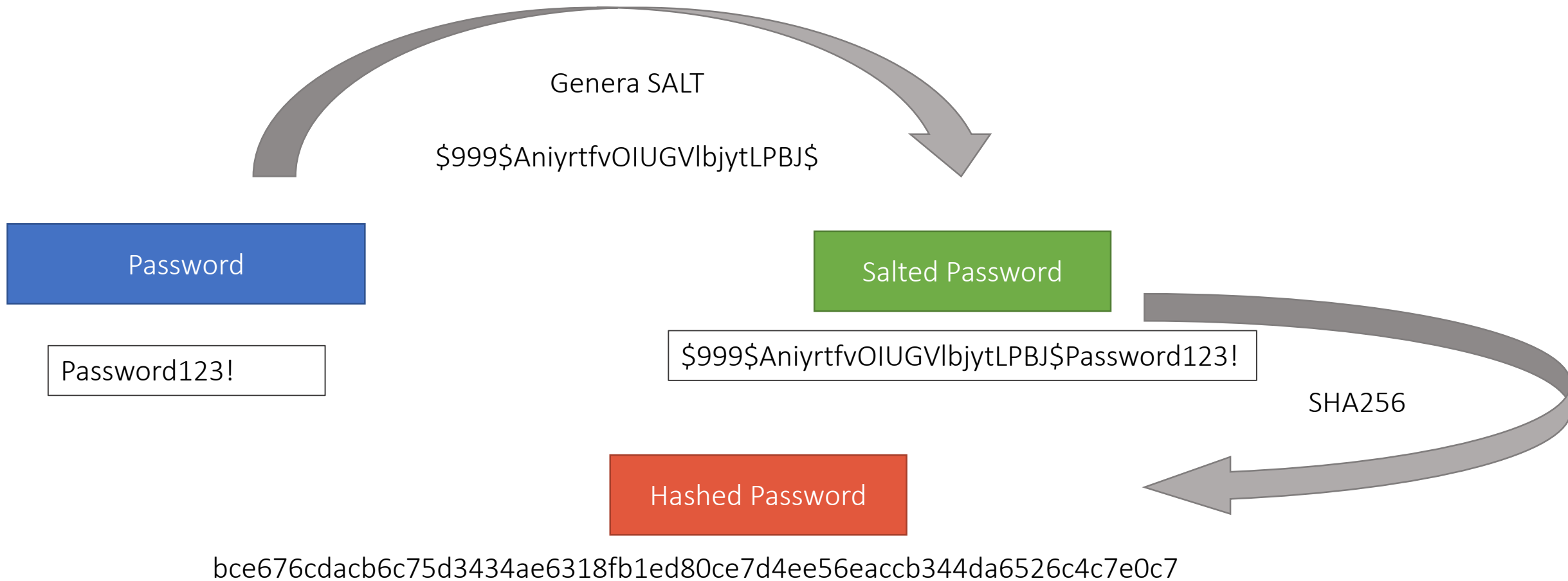
Scegliere una password complessa può essere d'aiuto per mitigare gli attacchi di password guess.

La complessità di una password è detta ***Entropia***

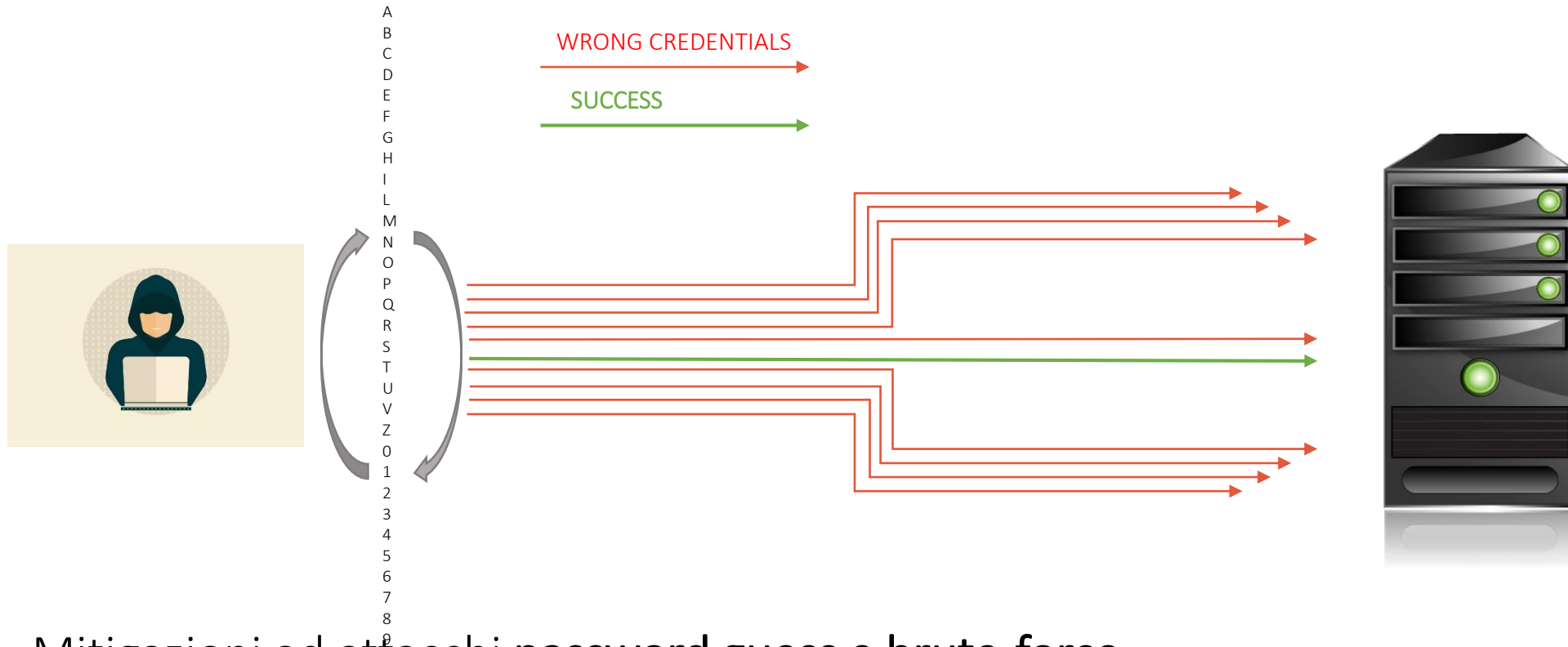
- ☐ *Lunghezza : più la password è lunga e meglio è*
- ☐ *Set di caratteri utilizzati : utilizzare combinazioni di set di caratteri diversi (lettere maiuscole/minuscole, numeri, simboli di punteggiatura e caratteri speciali)*

PASSWORDS

Salting + Hashing



PASSWORDS - ATTACCHI BRUTE FORCE E PASSWORD GUESS



Mitigazioni ad attacchi **password guess e brute-force**

- Limitare il numero di tentative di accesso
- Aggiungere un ritardo nella gestione delle richieste
- Blocco dell'account (efficace ma può causare facilmente Denial-of-service)

PASSWORDS - PROCEDURE DI RECUPERO/RISPRISTINO

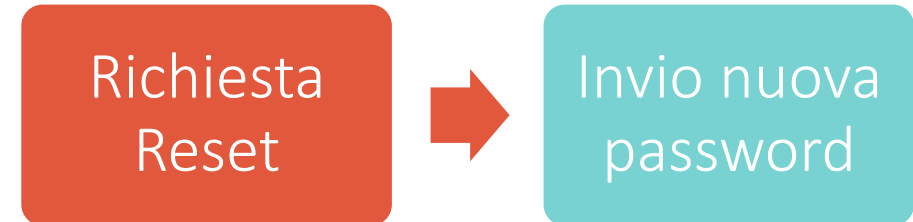
La maggior parte delle procedure di recupero password fanno affidamento sull'univocità dell'indirizzo e-mail inserito in fase di registrazione

Scenario 1: l'applicazione invia un link al form per l'inserimento della nuova password



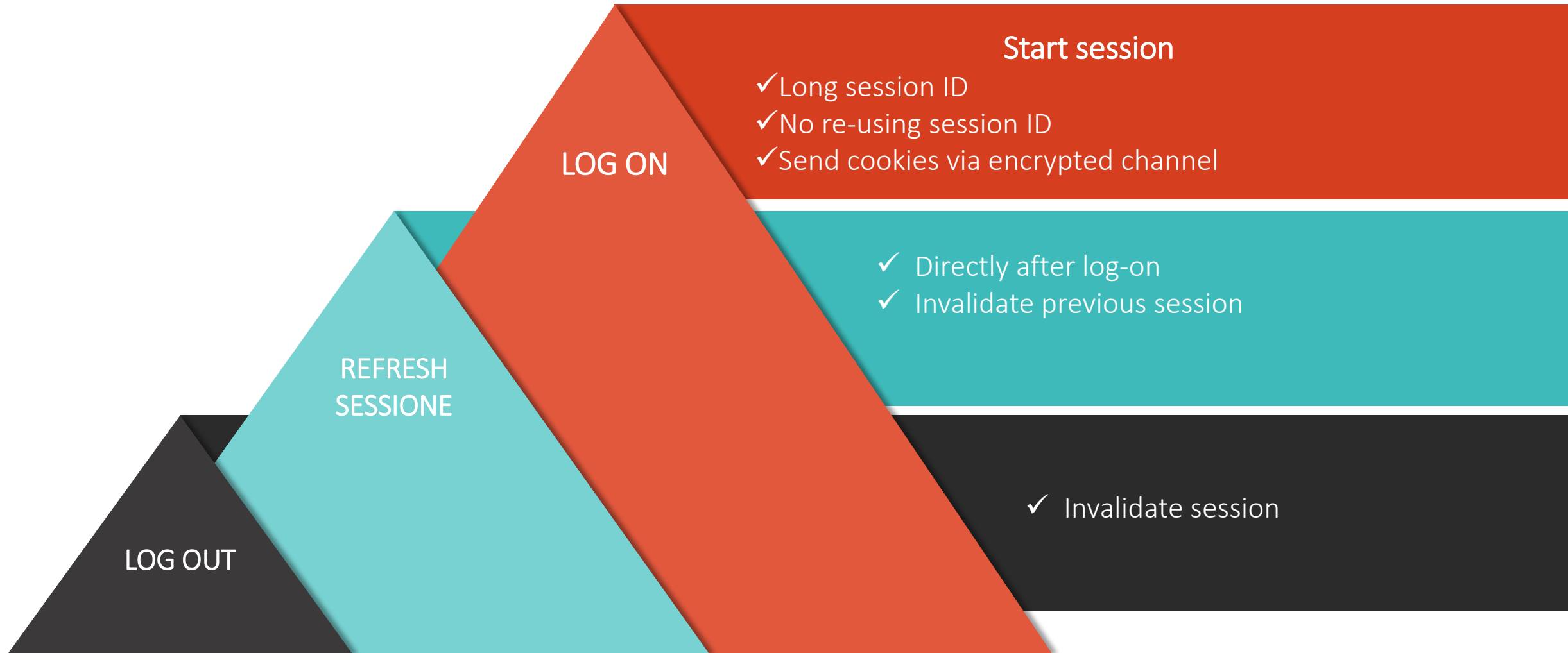
Cosa deve essere fatto per verificare l'identità dell'utente che richiede il reset della password cambia in base alla tipologia di applicazione
(ad es. forum vs home banking)

Scenario 2: l'applicazione invia la nuova password all'utente



Vantaggi: l'applicazione genera la nuova password e ne determina la robustezza.

Svantaggi: al 99% l'utente non ricorderà la password



- ✓ L'ID di sessione dovrebbe essere difficile da predire/indovinare. Generare un token abbastanza lungo
- ✓ Subito dopo il logout la sessione dovrebbe essere (immediatamente) invalidata
- ✓ La sessione dovrebbe essere invalidata dopo un certo tempo di inattività dell'utente (timeout di sessione)
- ✓ Gli ID di sessione non dovrebbero essere riutilizzati. Generando un token abbastanza lungo rende molto bassa la probabilità di riutilizz
- ✓ Immediatamente dopo un login, l'ID di sessione dovrebbe essere rinnovato e l'ID vecchio dovrebbe essere invalidato
- ✓ I cookie non dovrebbero essere usati, né tantomeno i parametri CGI
- ✓ I cookie di sessione dovrebbero viaggiare solo su canali criptati

A7:2021– Identification and Authentication Failures

Esempi di scenari di attacco

Scenario #1: Credential stuffing, l'uso di elenchi di password conosciute, è un attacco comune. Si supponga che un'applicazione non implementi la protezione automatizzata dalle minacce o dal password guessing. In tal caso, l'applicazione può essere utilizzata per determinare se le credenziali sono valide.

Scenario #2: la maggior parte degli attacchi di autenticazione si verifica a causa dell'uso continuato delle password come unico fattore. Una volta considerate le migliori pratiche, la rotazione delle password e i requisiti di complessità incoraggiano gli utenti a utilizzare e riutilizzare le password deboli. Si consiglia alle organizzazioni di interrompere queste pratiche secondo NIST 800-63 e di utilizzare l'autenticazione a più fattori.

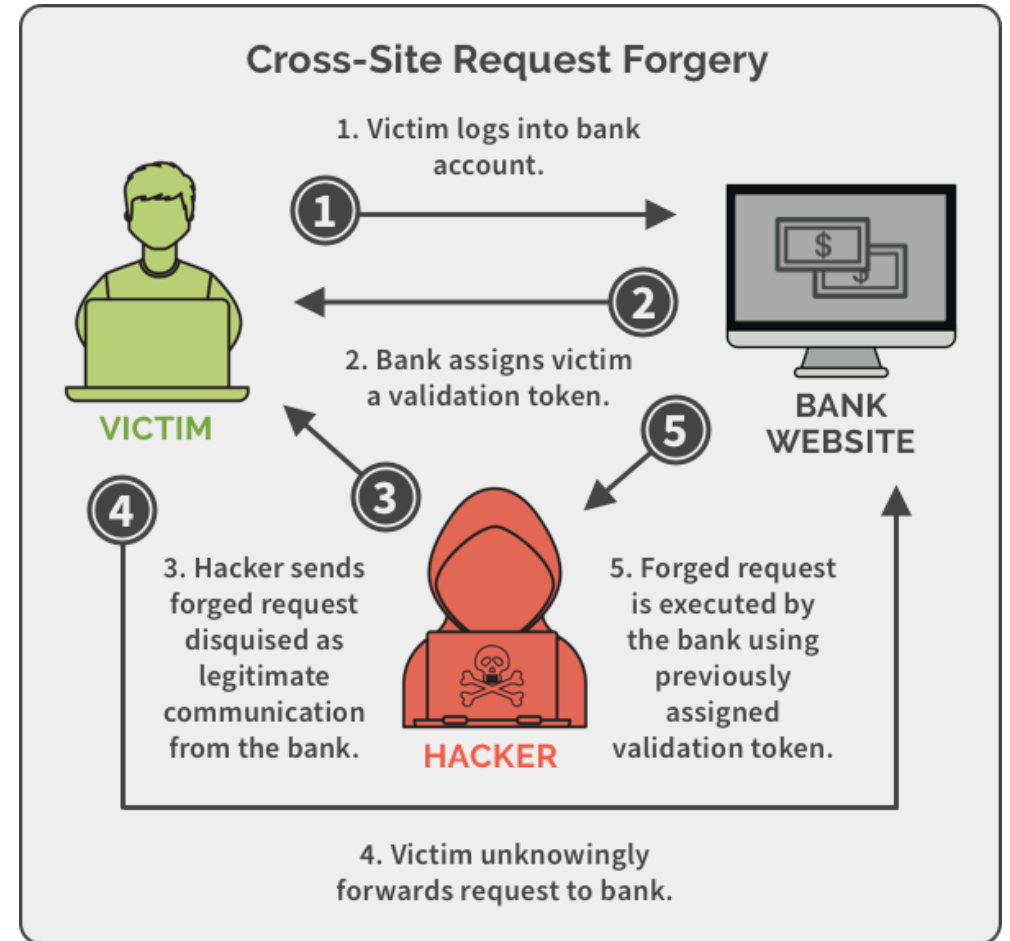
Scenario #3: i timeout della sessione dell'applicazione non sono impostati correttamente. Un utente utilizza un computer pubblico per accedere a un'applicazione. Invece di selezionare "logout", l'utente chiude semplicemente la scheda del browser e si allontana. Un utente malintenzionato utilizza lo stesso browser un'ora dopo e l'utente è ancora autenticato.

Cross Site Request Forgery (CSRF – XSRF)

CSRF è un attacco che obbliga gli utenti autenticati a inviare una richiesta a un'applicazione Web su cui sono attualmente autenticati senza esserne consapevoli.

Gli attacchi CSRF sfruttano la fiducia che un'applicazione Web ha in un utente autenticato.

Un attacco CSRF sfrutta una vulnerabilità in un'applicazione Web che non è in grado di distinguere tra una richiesta generata da un singolo utente in modo volontario e una richiesta generata da un utente senza il suo consenso.



Cross Site Request Forgery (CSRF – XSRF)

Mitigazione: rendere le richieste imprevedibili/attendibili

- 1) Lato server: generare un *nonce* o *CSRF-token* e memorizzarne il valore nei dati di sessione. Mandarlo al client
- 2) Lato client: includere il token in ogni richiesta (POST e, se possibile, GET)
- 3) Lato server: prima di gestire la richiesta verificare se il token inviato dal client corrisponde a quello presente in sessione.

+

Browser Same Origin Policy

Clickjacking - Mitigazione

Proviamo ad includere la pagina di login di un account Google in una pagina html usando un frame

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Esempio Frame</title>
5   <style>
6     .frame {
7       height:100vh;
8       width:100%;
9     }
10  </style>
11 </head>
12 <body>
13   <iframe class="frame"
14     src="https://accounts.google.com/signin/v2/identifier?hl=it&passive=true&continue=https%3A%2F%2Fwww.google.it%2F&ec=GAZAmgQ&flowName=GlifWebSignIn&flowEntry=ServiceLogin">
15   </iframe>
16 </body>
17 </html>
```



Firefox Can't Open This Page

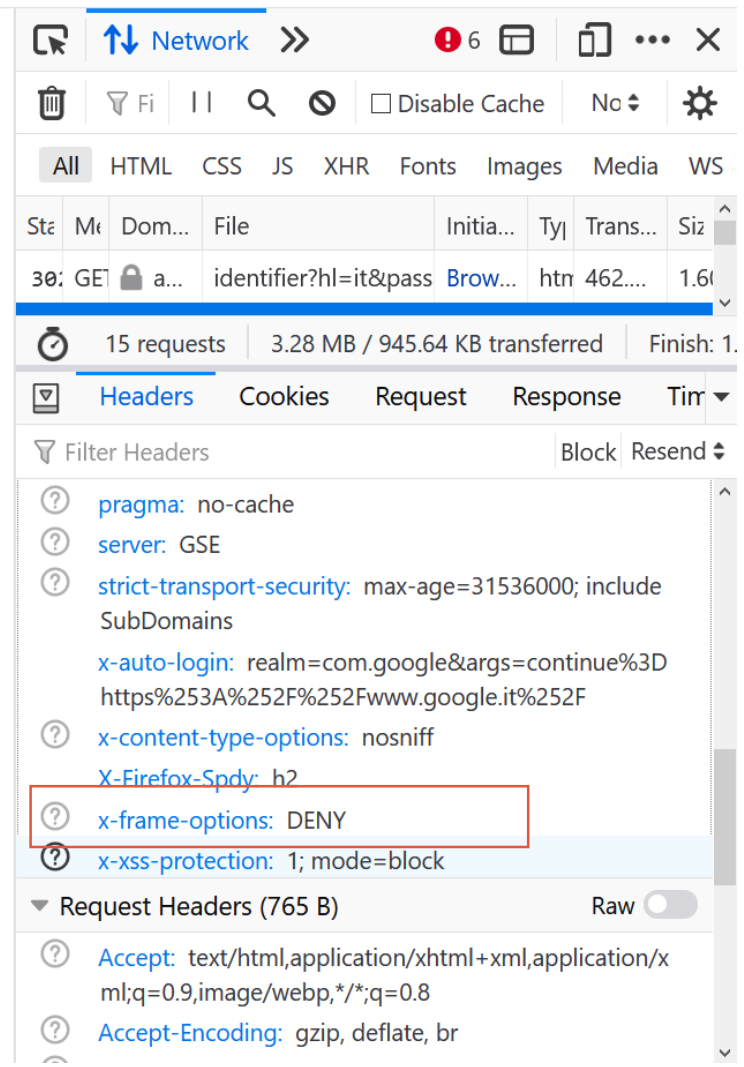
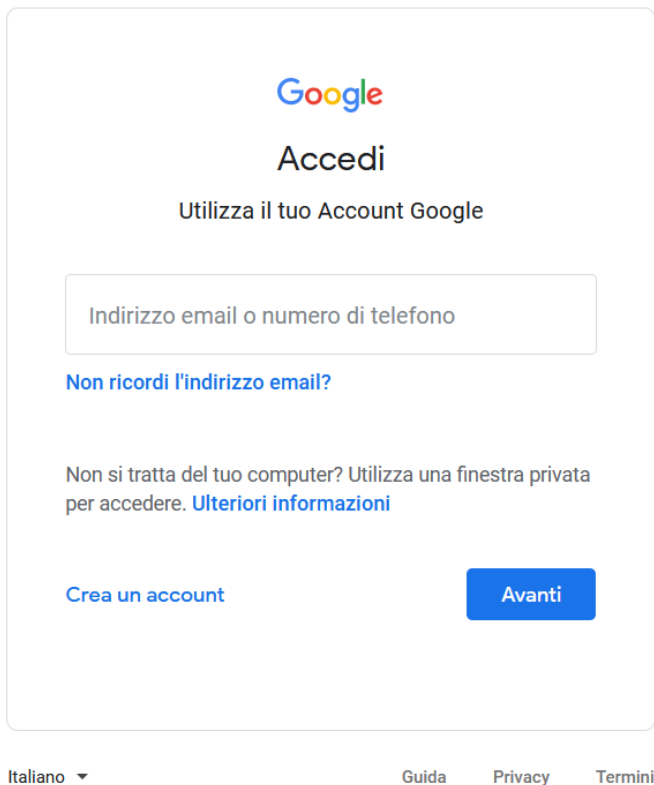
To protect your security, www.google.it will not allow Firefox to display the page if another site has embedded it. To see this page, you need to open it in a new window.

[Learn more...](#)

Open Site in New Window

☐ Report errors like this to help Mozilla identify and block malicious sites

Clickjacking - Mitigazione



Modulo III

Gestione dell'input utente

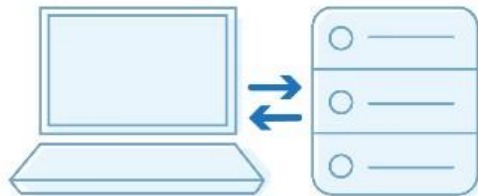
- Attacchi di injection
- Query dirette e parametrizzate
- Soluzioni per attacchi SQL injection
- Whitelist, blacklist e regular expressions
- Convalida dell'input
- Normalizzazione e codifica dell'input
- Buffer overflow e come influiscono sulla sicurezza
- Gli attacchi XSS e le mitigazioni

Injection Attacks - SQL Injection (SQLi)

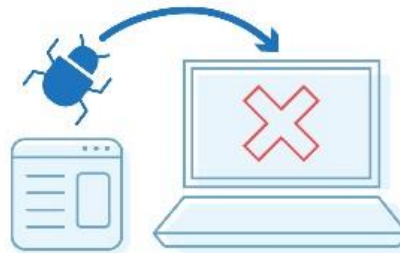
La vulnerabilità di injection esiste quando i dati manipolati vengono interpretati involontariamente da un sistema come un'istruzione anziché come semplici dati.

What Is SQL Injection

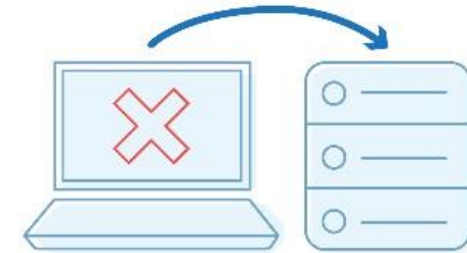
Applications can talk to a database using SQL queries.



SQL injection occurs when the application does not protect against malicious SQL queries..

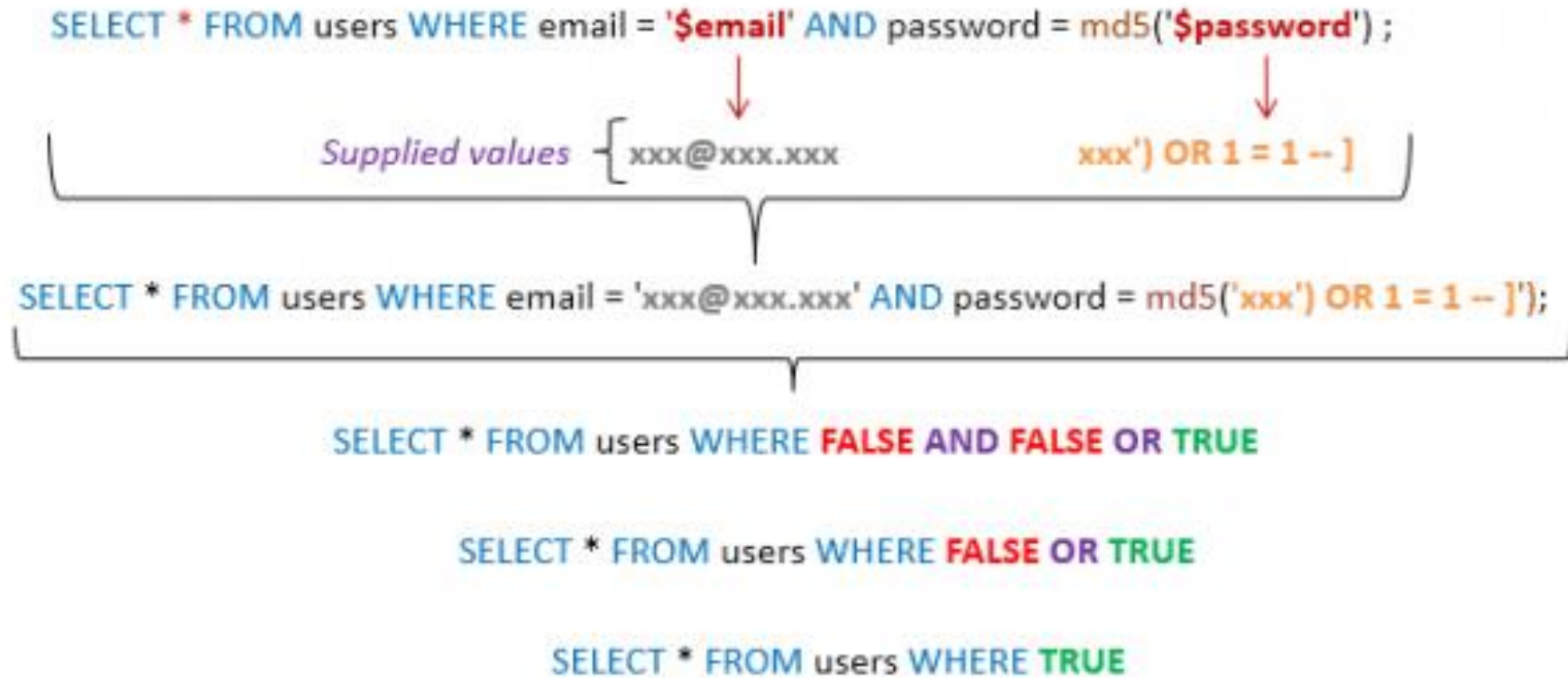


An attacker can use malicious SQL queries to trick the database into providing sensitive information.



Injection Attacks - SQL Injection (SQLi)

Un esempio



Injection Attacks - SQL Injection (SQLi)

Query dirette e query parametrizzate

Se analizzassimo l'istruzione come SQL e solo successivamente sostituissimo l'input dell'utente, non ci sarebbe alcuna possibilità di ingannare l'interprete SQL.

Il meccanismo detto parametrizzazione delle query fa esattamente questo:

- Innanzitutto, un'istruzione SQL che contiene segnaposto viene fornita alla routine del database, dove viene analizzata.
- Quindi l'input dell'utente viene sostituito in questi segnaposto.

La forma non parametrizzata delle query SQL è denominata query diretta. Le query parametrizzate sono talvolta chiamate **prepared statement**.

```
query = connection.prepare("SELECT * FROM people WHERE firstname  
= ?")  
connection.execute(query, $post.firstname)
```

Injection Attacks - Altre tipologie

SHELL injection

`site.web/getfile.php?file=legitfile.jpg;rm -rf *`

LDAP injection

String filter = "(&(USER = " + user_name + ") (PASSWORD = " + user_password + "))";

Attack: user_name=j.doe(& ---> (&(USER = j.doe(&))(PASSWORD=pass) Condizione sempre vera, bypassiamo il controllo sulla password

XML injection

Particolarmente efficace se si utilizzano le query XPath.

In questo caso è possibile utilizzare XQuery per parametrizzare le query (alla stregua delle query parametrizzate SQL)

NOSQL injection

Login Request Body: `username={$ne:1 }&psw={$ne1}`

`db.logins.find({ username: {$ne:1 }, password {$ne: 1 })`

Equivalente a: `SELECT * FROM logins WHERE username<>1 and password<>1`

Injection Attacks - Shell Injection

Supponiamo che un'applicazione prenda in input un indirizzo IP ed esegua il comando ping per vedere se l'host è raggiungibile, mostrando il risultato dell'operazione

```
ping -c 5 10.23.56.200
```



```
PING 10.23.56.200 (10.23.56.200) 56(84) bytes of data.  
64 bytes from 10.23.56.200 : icmp_seq=1 ttl=64 time=0.023 ms  
64 bytes from 10.23.56.200 : icmp_seq=2 ttl=64 time=0.074 ms  
64 bytes from 10.23.56.200 : icmp_seq=3 ttl=64 time=0.074 ms  
64 bytes from 10.23.56.200 : icmp_seq=4 ttl=64 time=0.072 ms  
64 bytes from 10.23.56.200 : icmp_seq=5 ttl=64 time=0.037 ms
```

Se un attacker concatena all'indirizzo ip il valore `;id` visualizzerebbe informazioni sull'utente che esegue il comando ping

```
ping -c 5 10.23.56.200; id
```



```
PING 10.23.56.200 (10.23.56.200) 56(84) bytes of data.  
64 bytes from 10.23.56.200 : icmp_seq=1 ttl=64 time=0.023 ms  
64 bytes from 10.23.56.200 : icmp_seq=2 ttl=64 time=0.074 ms  
64 bytes from 10.23.56.200 : icmp_seq=3 ttl=64 time=0.074 ms  
64 bytes from 10.23.56.200 : icmp_seq=4 ttl=64 time=0.072 ms  
64 bytes from 10.23.56.200 : icmp_seq=5 ttl=64 time=0.037 ms
```

```
--- 10.23.56.200 ping statistics ---  
5 packets transmitted, 5 received, 0% packet loss, time 3999ms  
rtt min/avg/max/mdev = 0.023/0.056/0.074/0.021 ms  
uid=0(root) gid=0(root) groups=0(root)
```

A3:2021 - Injection

Threat Agents / Attack Vectors	Security Weakness	Impacts
Quasi tutte le fonti di dati possono essere un vettore di injection, variabili di ambiente, parametri, servizi Web esterni e interni e tutti i tipi di utenti. I difetti di injection si verificano quando un utente malintenzionato può inviare dati ostili a un interprete.	I difetti di injection sono molto diffusi, in particolare nel codice legacy. Le vulnerabilità di injection si trovano spesso nelle query SQL, LDAP, XPath o NoSQL, nei comandi del sistema operativo, nei parser XML, nelle intestazioni SMTP, nei linguaggi di espressione e nelle query ORM. I difetti di injection sono facili da scoprire durante l'esame del codice. Scanner e fuzzer possono aiutare gli aggressori a trovare tali vulnerabilità.	L'injection può comportare la perdita di dati, il danneggiamento o la divulgazione a parti non autorizzate, la perdita di responsabilità o il rifiuto di accesso. A volte può portare alla completa acquisizione dell'host. L'impatto sul business dipende dalle esigenze dell'applicazione e dei dati.

COS'E'

- “Untrusted data” inviati all’interprete
 - command / query
 - headers
 - cookies
 - {.. Altre forme di input..}
- Interprete esegue input come comando

CHI E' SUSCETTIBILE

- SQL
- Hadoop
- SOAP
- XML
- {.. Everything ..}

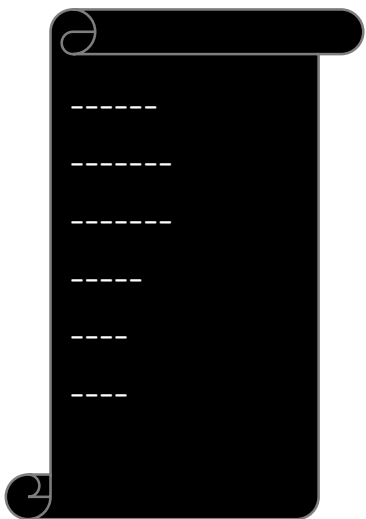
- Mancata separazione fra “Untrusted data” e comandi
- Mancanza di input validation/sanitization
- Attacker può modificare l’execution Context

A3:2021 - Injection

Come prevenire

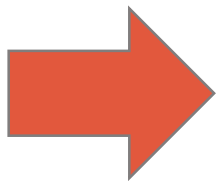
- Prevenire l'iniezione richiede di mantenere i dati separati da comandi e query.
- L'opzione preferita è utilizzare un'API sicura, che evita completamente l'uso dell'interprete o fornisce un'interfaccia parametrizzata, oppure eseguire la migrazione per utilizzare ORM (Object Relational Mapping Tools).
- Nota: anche se parametrizzate, le stored procedure possono comunque introdurre SQL injection se PL / SQL o T-SQL concatena query e dati o esegue dati ostili con EXECUTE IMMEDIATE o exec ().
- Utilizzare la convalida dell'input lato server positiva o "whitelist". Questa non è una difesa completa poiché molte applicazioni richiedono caratteri speciali, come aree di testo o API per applicazioni mobili.
- Per qualsiasi query dinamica residua, esegui l'escape dei caratteri speciali utilizzando la sintassi di escape specifica per quell'interprete.
- Nota: la struttura SQL come i nomi di tabella, i nomi di colonna e così via non possono essere sottoposti a escape e quindi i nomi di struttura forniti dall'utente sono pericolosi. Questo è un problema comune nel software di scrittura di report.
- Utilizzare LIMIT e altri controlli SQL all'interno delle query per impedire la divulgazione di massa dei record in caso di iniezione SQL.

Validazione dell'input - Filtering



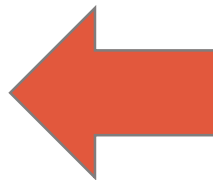
Blacklist approach

Rifiutiamo qualsiasi
valore contenuto
nella lista



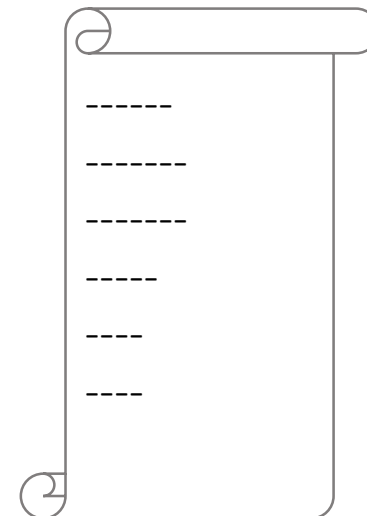
```
Char[] bList=['', ' ', '/', '.', ...];  
Foreach( char c in bList)  
    if(input.indexOf(c)>-1)  
        retur false;
```

```
String wList=«a,b,c, ...,0,1,2, ..., 9»;  
Foreach(char c in input)  
    if(!wList.contains(c))  
        return false;
```



Whitelist approach

Accettiamo solo i
valori contenuti
nella lista



Validazione dell'input - Filtering e Regular Expressions

Usate per trovare pattern all'interno di stringhe, utili per la realizzazione compatta di blacklist / whitelist

HTML5 email field validation RegEx

```
/^[a-zA-Z0-9.!#$%&'*/+=?^_`{|}~-]+@[a-zA-Z0-9-]+(?:\.[a-zA-Z0-9-]+)*$/
```

Esempio JAVA RegEx data

Formato: g/m/a
Range: 01/01/1900 – 31/12/2099

```
(0?[1-9]|[12][0-9]|3[01])/(0?[1-9]|1[012])/((19|20)\\d\\d)
```

```
(  
    #start of group #1  
    0?[1-9]      # 01-09 or 1-9  
    |           # ..or  
    [12][0-9]    # 10-19 or 20-29  
    |           # ..or  
    3[01]        # 30, 31  
    )           #end of group #1  
    /          # follow by a "/"  
    (  
        # start of group #2  
        0?[1-9]  # 01-09 or 1-9  
        |       # ..or  
        1[012]   # 10,11,12  
        )       # end of group #2  
        /       # follow by a "/"  
        (  
            # start of group #3  
            (19|20)\\d\\d  # 19[0-9][0-9] or 20[0-9][0-9]  
        )       # end of group #3  
    )
```

Validazione dell'input

Sanitization, encoding e normalization, misinterpretation

Sanitization: «ripulire» l'input utente prima che venga dato in input ai processi per la sua elaborazione

Encoding: «trasformare» l'input/output prima che venga utilizzato o visualizzato all'utente. L'encoding è Context Sensitive

Normalization: utilizzare la stessa codifica per determinati caratteri che utilizzano encoding differenti per rappresentare la stessa informazione. Ad esempio il carattere < ha diverse rappresentazioni in HTML

```
&lt; &#x3c; &#x3C; &#x003c; &#60; &#0060; &#060; &#00000000000060;
```

Misinterpretation: interpretazione diversa di uno stesso input in parti diverse di un applicazione

Modulo IV

Autorizzazione

- Le dimensioni dell'autorizzazione
- Riferimento diretto e indiretto ad oggetti
- Session poisoning and race conditions.

Verificare se a qualcuno o qualcosa è permesso compiere una determinata azione

Misura contro **PRIVILEGE ESCALATION**

Autenticazione

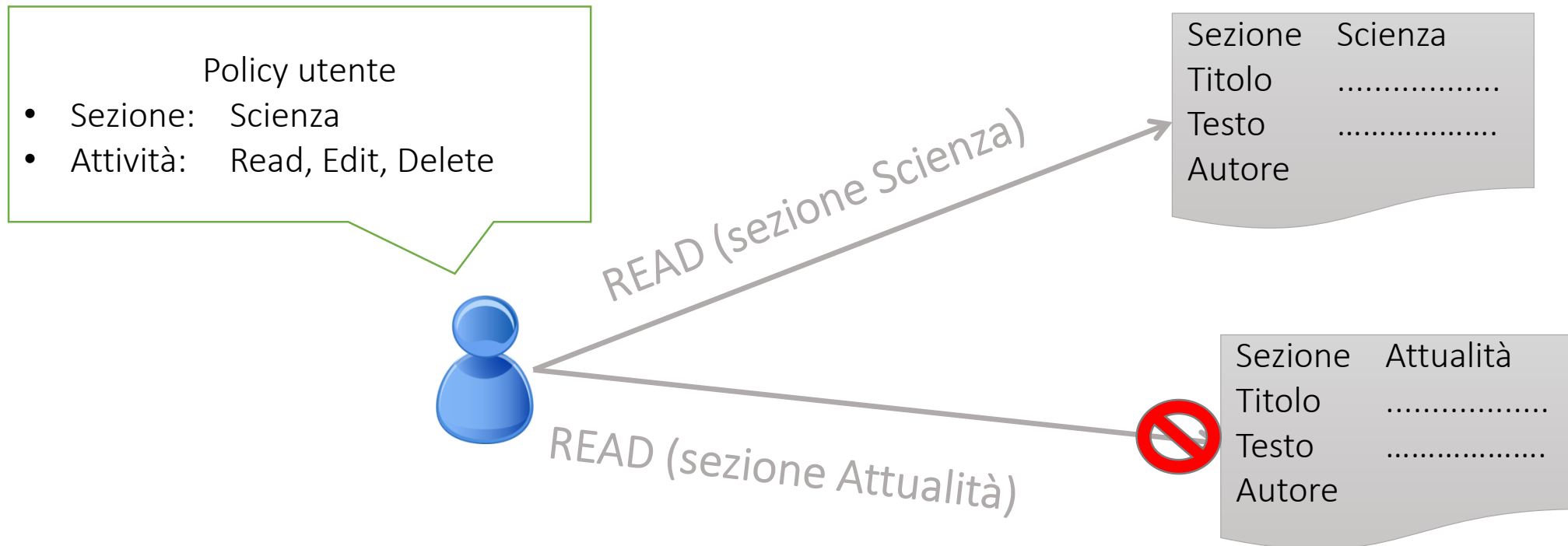
Verificare l'identità del
soggetto

Autorizzazione

Verificare cosa il soggetto
autenticato può fare

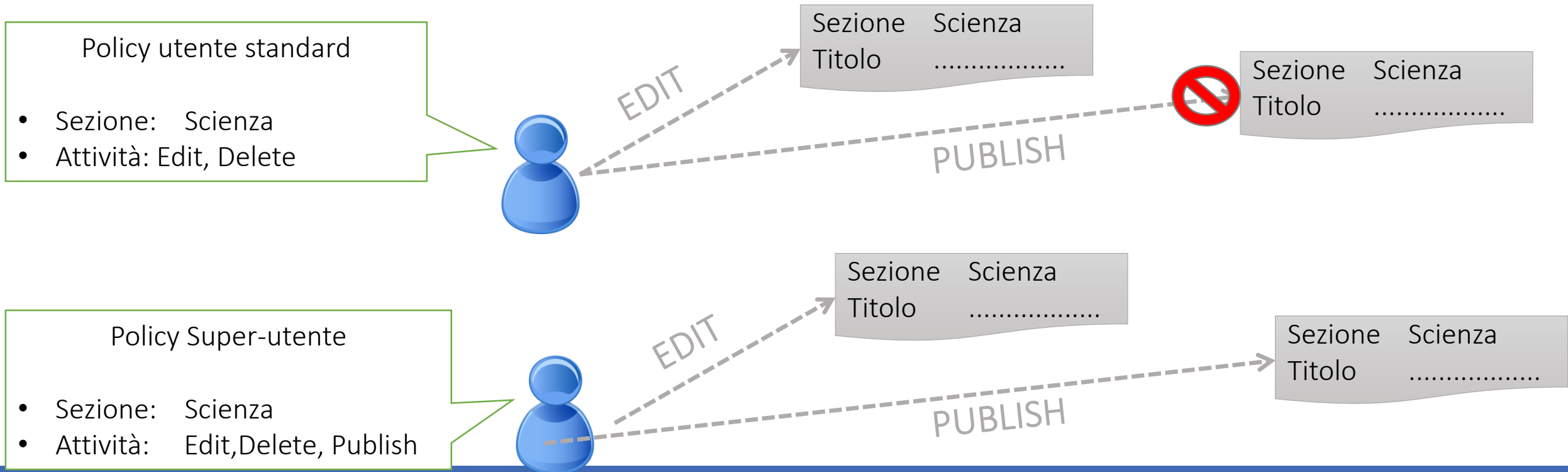
Autorizzazione orizzontale

Verificare se una determinata operazione può essere effettuata su un oggetto indipendentemente da chi la compie



Autorizzazione verticale

Verificare se qualcuno ha il permesso di effettuare una determinata azione indipendentemente dall'oggetto su cui l'azione avrà effetto



RISCHI

- I controlli di autorizzazione sono dati spesso per scontato al punto che non vengono implementati, soprattutto quelli relativi all'autorizzazione orizzontale. Ciò causa utilizzo di risorse a cui non sarebbe consentito accedere

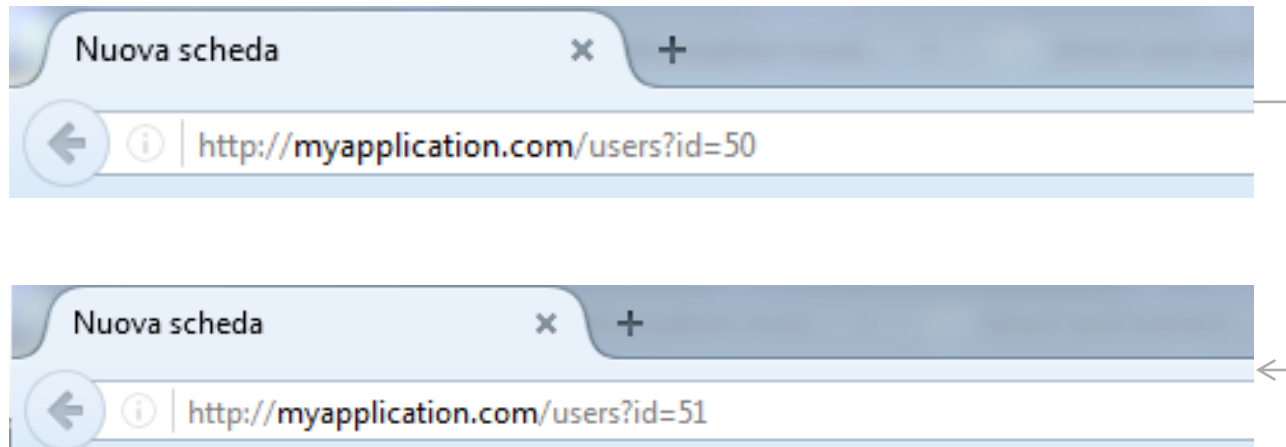
MITIGAZIONI

- Implementare un meccanismo centralizzato di controlli di autorizzazione
- Utilizzare il principio del **LEAST PRIVILEGE**: eseguire il codice con il più basso livello di privilegi possibile
- Utilizzare tecniche di riferimento indiretto

Riferimento diretto e indiretto

Definizione OWASP

Un riferimento diretto a un oggetto si verifica quando uno sviluppatore espone un riferimento a un oggetto di implementazione interna, come un file, una directory, un record di database o una chiave, come un URL o un parametro del modulo. Un utente malintenzionato può manipolare riferimenti diretti a oggetti per accedere ad altri oggetti senza autorizzazione, a meno che non sia in atto un controllo di controllo dell'accesso



1. L'utente con id=50 accede alla propria pagina di profilo
2. L'utente cambia il parametro id
3. L'utente accede alla pagina di profilo di un altro utente

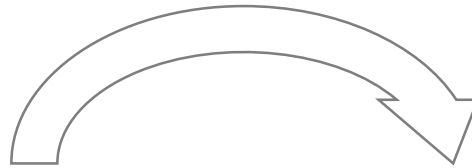
Riferimento diretto e indiretto

Riferimento indiretto

Accediamo ad un oggetto tramite un puntamento (chiave, indice)

COLLECTION USERS

ID	UserName
50	John Doe
51	Sam Smith
.....	



SESSION COLLECTION USERS

KEY	ID	UserName
64ECSD-HSDY	50	John Doe
NKH823-MHAS	51	Sam Smith
		

Utilizzando una chiave o indice che punta all'elemento a cui si vuole fornire l'accesso l'attacker ha meno possibilità di successo

Race conditions - Attacco TOC TOU

Invoke di altre funzionalità applicative o metodi

Software Logic

Time Of Check

Func

Time Of Use

Funzionalità atte al controllo delle autorizzazioni dell'utente

Momento in cui si consuma effettivamente la risorsa

```
if (!access("/tmp/mydata.txt",W_OK))
{
    // TOC: check if we have
    write permission
    // ...
    // ATTACK WINDOW
    f =
    fopen("/tmp/mydata.txt","w+");
    // TOU: open the file for
    writing
    write_data_to_file(f);
} else {
    fprintf(stderr,"Unable to
    open file!\n");
}
```

Implementare TOC TOU in operazioni atomiche limita il rischio di accesso non autorizzato

Session Poisoning

- A volte, un flusso di lavoro in un'applicazione Web è suddiviso in più passaggi.
- Se questi passaggi vengono eseguiti in modo anomalo e il codice non è affidabile in una situazione del genere, le cose potrebbero andare storte.
- Ciò può causare un problema di sicurezza se i controlli di sicurezza, come i controlli di integrità o di autorizzazione, possono essere aggirati.
- In un flusso di lavoro, i risultati intermedi vengono spesso archiviati nei dati della sessione. **Di solito si presume che i dati della sessione siano attendibili.**

Session Poisoning

Scenario di esempio

- Un cliente seleziona gli articoli da inserire nel carrello e quindi procede alla pagina di pagamento.
- Nella pagina di checkout, viene calcolato l'importo totale e le spese di spedizione sono determinate in base al numero totale di articoli.
- Questi importi vengono memorizzati nei dati di sessione dell'applicazione. Tuttavia, prima di accettare il pagamento, il cliente aggiunge altri articoli al carrello riproducendo la richiesta di "aggiungi al carrello".
- Quindi, il cliente richiede la pagina di pagamento finale, saltando la pagina di checkout e il ricalcolo dei costi totali e di spedizione. Se i dati della sessione contengono ancora gli importi precedenti, il cliente paga meno di quanto dovrebbe.

Modulo V

Configurazione, gestione degli errori e logging

- Hardening
- Information leaks
- logging
- Il principio di 'Fail Securely'.
- Attacchi Denial of Service e mitigazioni

Hardening

Mettere il software in sicurezza in modo diverso dalla programmazione.
Irrobustire le difese dei sistemi su cui il software è in esecuzione poichè non si è a conoscenza del livello di sicurezza delle componenti di terze parti utilizzate dalle nostre applicazioni

Applicare le ultime patch di sicurezza

Restringere l'accesso a funzionalità sensibili o pericolose
(ad es. il firewall)

Configurare i privilegi di accesso a file, servizi, applicazioni

Disabilitare o rimuovere le funzionalità di debug

Rimuovere tutti i file non necessari

Cambiare le credenziali di default

Come verifico se le configurazioni effettuate sul sistema sono efficaci?

1. Sviluppatori e sistemisti devono comunicare per capire se e cosa è stato fatto a livello di configurazione del sistema
2. Utilizzare vulnerability scanners

A5:2021 – Security Misconfiguration

Threat Agents / Attack Vectors	Security Weakness	Impacts
Gli aggressori tenteranno spesso di sfruttare i difetti senza patch o di accedere ad account predefiniti, pagine inutilizzate, file e directory non protetti, ecc. Per ottenere accesso non autorizzato o conoscenza del sistema.	L'errata configurazione della sicurezza può verificarsi a qualsiasi livello di uno stack di applicazioni, inclusi servizi di rete, piattaforma, server Web, server applicazioni, database, framework, codice personalizzato e macchine virtuali, contenitori o storage preinstallati. Gli scanner automatici sono utili per rilevare errori di configurazione, utilizzo di account o configurazioni predefiniti, servizi non necessari, opzioni legacy, ecc.	Tali difetti spesso danno agli aggressori l'accesso non autorizzato ad alcuni dati o funzionalità di sistema. Occasionalmente, tali difetti comportano un completo compromesso del sistema. L'impatto sul business dipende dalle esigenze di protezione dell'applicazione e dei dati.

- Configurazioni predefinite non sicure
 - Configurazioni incomplete o ad hoc
 - Storage cloud aperta
 - Intestazioni HTTP configurate in modo errato e messaggi di errore dettagliati contenenti informazioni sensibili
- Funzionalità non necessarie abilitate
 - Account di default abilitati
 - Errata gestione degli errori
 - Ultime patch di sicurezza non installate
 - Software non aggiornato

A5:2021 – Security Misconfiguration

Come prevenire

È necessario implementare processi di installazione sicura, tra cui:

- Un processo di rafforzamento ripetibile che semplifica e velocizza la distribuzione di un altro ambiente bloccato. Gli ambienti di sviluppo, controllo qualità e produzione devono essere tutti configurati in modo identico, con credenziali diverse utilizzate in ogni ambiente. Questo processo dovrebbe essere automatizzato per ridurre al minimo lo sforzo richiesto per configurare un nuovo ambiente sicuro.
- Una piattaforma minima senza funzionalità, componenti, documentazione ed esempi non necessari. Rimuovere o non installare funzionalità e framework inutilizzati.
- Un'attività per rivedere e aggiornare le impostazioni appropriate a tutte le note di sicurezza, gli aggiornamenti e le patch come parte del processo di gestione delle patch (vedi A9: 2017-Utilizzo di componenti con vulnerabilità note). In particolare, rivedi le autorizzazioni di archiviazione cloud (ad es. Autorizzazioni del bucket S3).
- Un'architettura applicativa segmentata che fornisce una separazione efficace e sicura tra componenti o tenant, con segmentazione, containerizzazione o gruppi di sicurezza cloud (ACL).
- Invio di direttive di sicurezza ai client, ad es. Intestazioni di sicurezza.
- Un processo automatizzato per verificare l'efficacia delle impostazioni e delle impostazioni in tutti gli ambienti.

A5:2021 – Security Misconfiguration

Esempi di scenari di attacco

Scenario #1: il server delle applicazioni viene fornito con applicazioni di esempio non rimosse dal server di produzione. Queste applicazioni di esempio presentano falle di sicurezza note utilizzate dagli aggressori per compromettere il server. Supponiamo che una di queste applicazioni sia la Console di amministrazione e che gli account predefiniti non siano stati modificati. In tal caso, l'attaccante accede con password predefinite e prende il sopravvento.

Scenario #2: l'elenco delle directory non è disabilitato sul server. Un utente malintenzionato scopre che può semplicemente elencare le directory. L'attaccante trova e scarica le classi Java compilate, che decompila e decodifica per visualizzare il codice. L'autore dell'attacco trova quindi un grave difetto di controllo dell'accesso nell'applicazione.

Scenario #3: la configurazione del server delle applicazioni consente di restituire agli utenti messaggi di errore dettagliati, ad esempio tracce dello stack. Ciò espone potenzialmente informazioni riservate o difetti sottostanti come versioni di componenti note per essere vulnerabili.

Scenario #4: un provider di servizi cloud (CSP) dispone di autorizzazioni di condivisione predefinite aperte a Internet da altri utenti CSP. Ciò consente di accedere ai dati sensibili archiviati nel cloud storage.

Information leaks aiutano un aggressore a migliorare i propri attacchi

Attenzione a ciò che scriviamo, memorizziamo, mostriamo e inviamo con le nostre applicazioni

- HTTP response headers contengono informazioni di servizi e versione, permettendo la ricerca di vulnerabilità note.
- Pagine di errore che mostrano l'intero stack di eccezione forniscono suggerimenti per migliorare gli attacchi
- Indirizzi IP locali e percorsi di cartelle o file sul server aiutano a prendere di mira altri obiettivi una volta compromesso un sistema oppure se l'aggressore usa il browser di qualcun'altro per effettuare attacchi dall'interno.
- Commenti nelle pagine HTML che mostrano nomi, credenziali, indirizzi email aiutano ad effettuare attacchi di **social engineering**.
- File di gestione di versione (git, svn ecc..) possono contenere del codice sorgente o informazioni relative.

MITIGATIONS

- ✓ Non utilizzare i commenti nel codice HTML perché sono renderizzati con la pagina
- ✓ Rimuovere informazioni di versione da qualsiasi cosa venga inviato dall'applicazione (ma non sempre è possibile)
- ✓ Utilizzare messaggi di errore generici in ambienti di produzione
- ✓ Non utilizzare mai indirizzi email personali (ad es. `info@mysite.com` oppure `webmaster@mysite.com`)

FAIL SECURE : Non lasciare il sistema in uno stato vulnerabile.

In che modo?

- Utilizzare i blocchi try-catch-finally se il linguaggio utilizzato lo permette
- Liberare ogni risorsa utilizzata e abbassare i privilegi di esecuzione
- Non mostrare i dettagli degli errori in ambienti di produzione per evitare che alcune informazioni possano essere utilizzate da un attacker

LOGGING

Aggiungere sempre le funzionalità di logging alle applicazioni, è utile per diagnosticare i problemi ed individuare eventuali attacchi

MA SOLO SE SI LOGGANO LE INFORMAZIONI CORRETTE

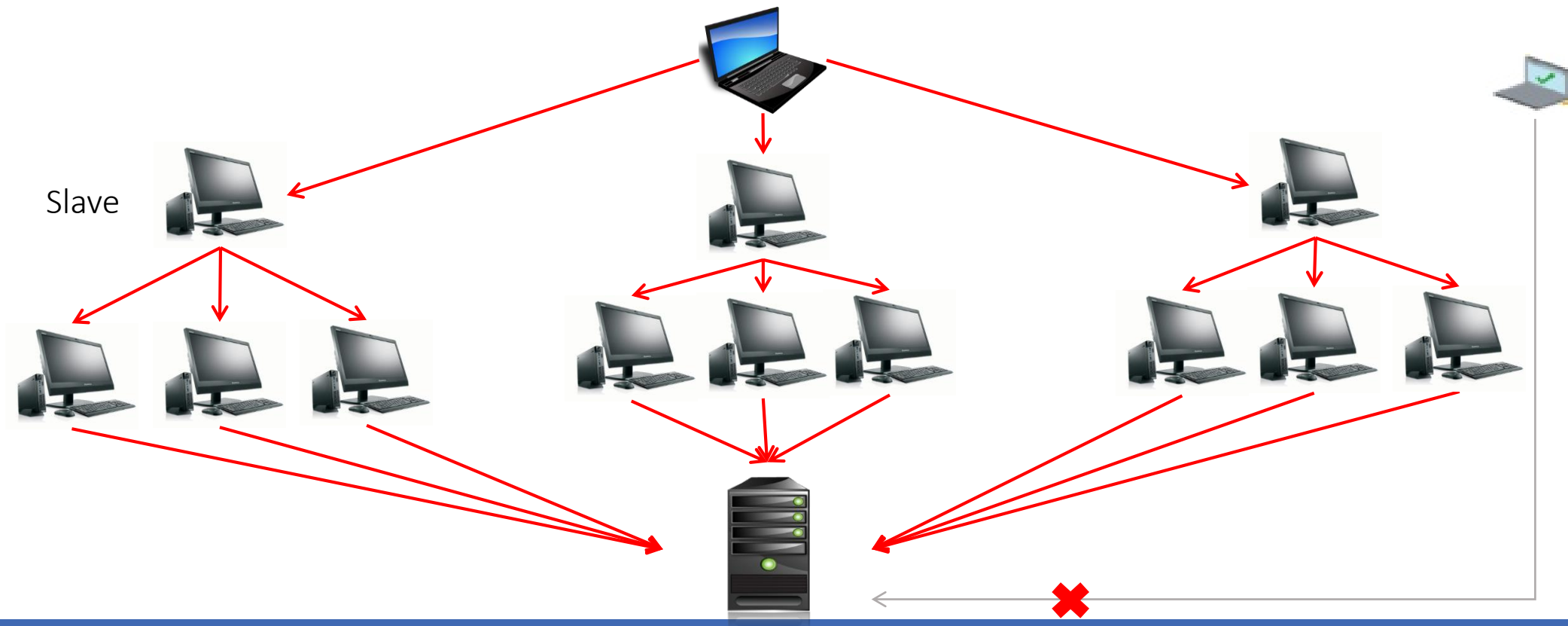
- ✓ Alcune informazioni non possono essere salvate in base alle normative sulla privacy, spesso specifiche per paese
- ✓ Le informazioni sensibili non dovrebbero essere loggate (password, carte di credito ecc..)
- ✓ Inviare un “heartbeat “ per assicurarsi che la funzionalità di logging sia attiva
- ✓ Ogni elemento loggato deve includere data e ora
- ✓ Ispezionare regolarmente I log per individuare eventuali anomalie
- ✓ Proteggere i log per evitare il tampering (write-once,read-many media, checksum)
- ✓ Loggare ogni elevazione di privilegio e ripristino allo stato precedente

Denial of Service Distribuiti (DDOS)

Attacco mirato per causare Denial of Service che di solito utilizza una botnet che esegue simultaneamente richieste verso un sistema per consumarne le risorse

Mitigazioni:

- Limitare l'utilizzo di risorse che possono essere allocate
- Applicare meccanismi di ritardo delle richieste



Modulo VI

Crittografia

- Il principio di Kerckhoffs
- Gestione delle chiavi e randomicità
- Crittografia simmetrica e asimmetrica
- Attacchi Man-In-The-Middle
- Infrastruttura a chiave pubblica e certificati
- HTTPS

Principio di Kerckoff

**A GOOD CRYPTOGRAPHIC ALGORITHM SHOULD NOT
DEPEND ON THE SECRECY OF THE ALGORITHM ITSELF.
THE ONLY PART THAT NEED TO KEEP SECRET IS THE KEY**

Scegliere sempre un algoritmo ben noto, ampiamente revisionato e analizzato di cui sia stata comprovata la sicurezza

Spesso la scelta di algoritmi open-source risulta essere la soluzione migliore

- Best practices

- ✓ Consultare sempre un esperto in materia quando si progetta il sistema
- ✓ Non inventarsi un proprio algoritmo, sicuramente sarà aggirato
- ✓ Fare attenzione alla gestione delle chiavi
 - ✓ Dove sono memorizzate (never with encrypted data)
 - ✓ Il livello di robustezza necessaria
 - ✓ Come si gestiscono le chiavi compromesse
 - ✓ Come vengono distribuite le chiavi
- ✓ Se è necessaria la generazione di sequenze casuali: scegliere un potente generatore di numeri casuali e utilizzare una sorgente con sufficiente entropia (usare le classi **secure random**)

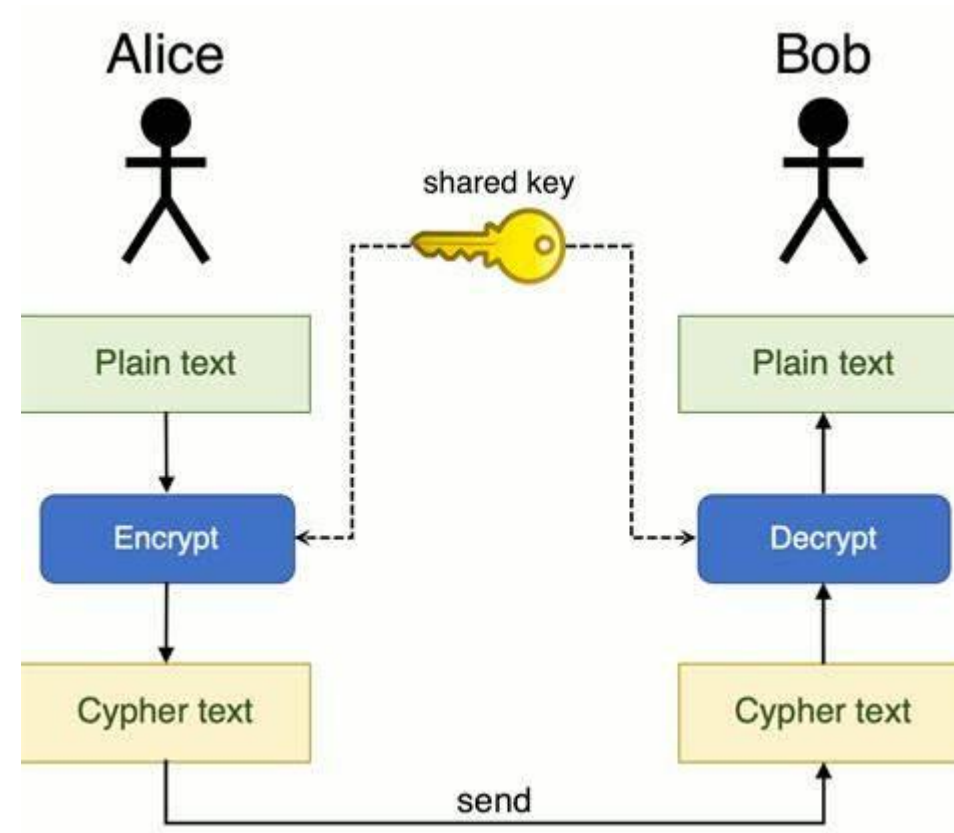
Crittografia simmetrica

Sender e Recipient utilizzano la stessa chiave condivisa per criptare e decriptare il messaggio.

La chiave condivisa deve essere scambiata prima dell'inizio della comunicazione e deve rimanere segreta.

Come fare a scambiare la chiave in sicurezza?

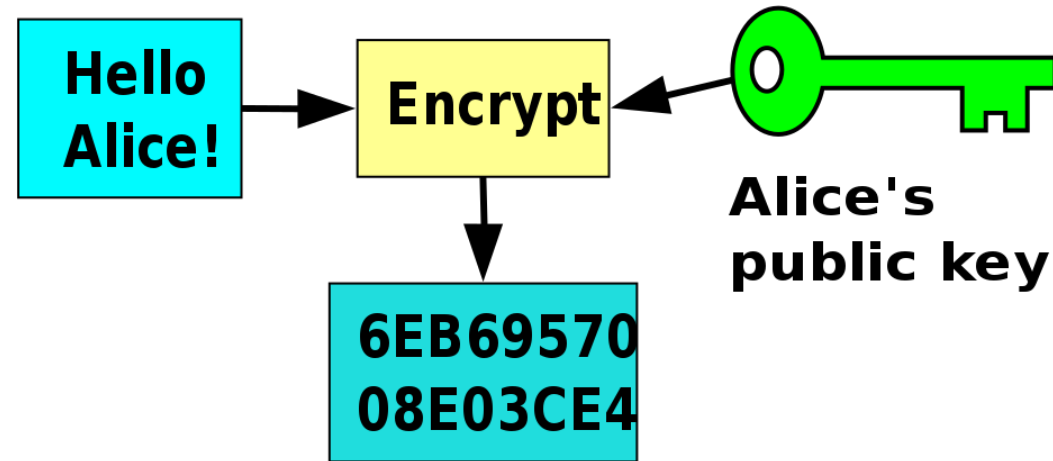
- Per questo scambio deve essere utilizzato un canale sicuro separato. Ad esempio si potrebbe scambiare la chiave di persona o via email. Quanto debba essere sicuro il canale dipende dal valore di ciò che si deve proteggere



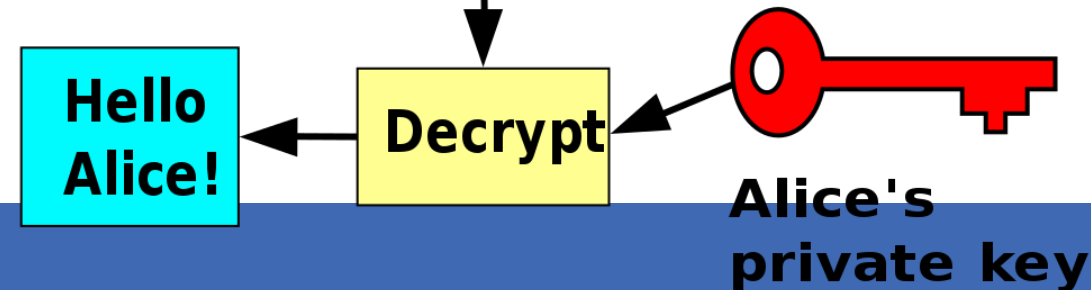
Crittografia asimmetrica (o a chiave pubblica)

Per inviare un messaggio criptato ad Alice, Bob deve criptare il messaggio con la chiave pubblica di Alice. Il messaggio può essere decriptato solo con la chiave privata di Alice, che si suppone possenga solo lei

Bob



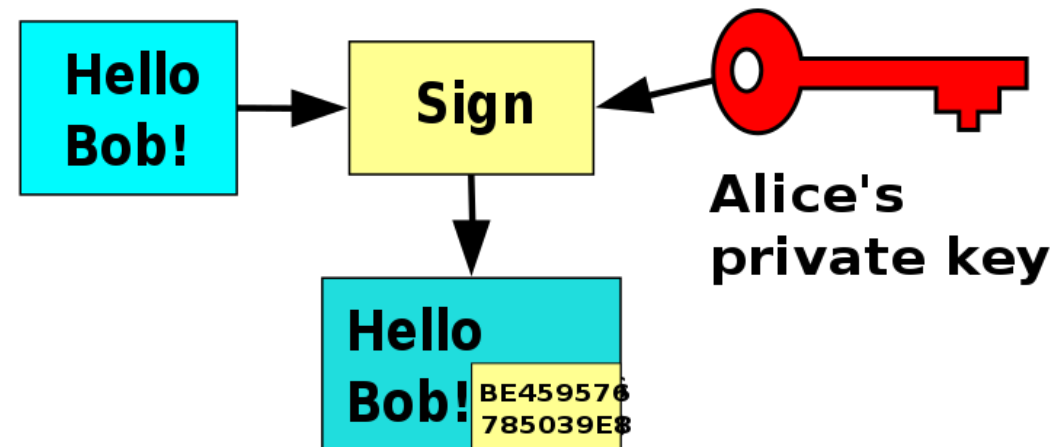
Alice



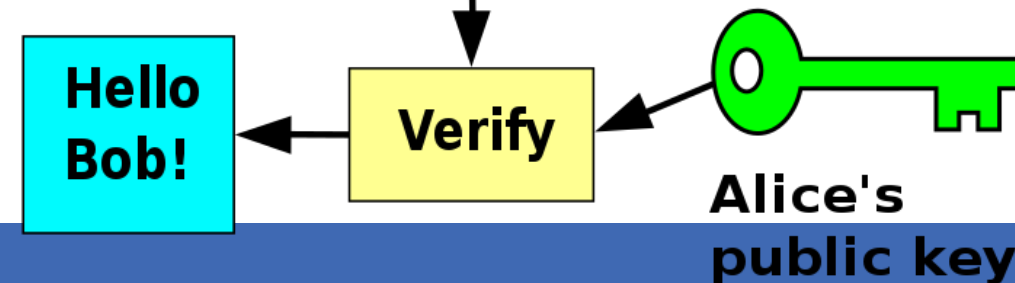
Crittografia asimmetrica (o a chiave pubblica)

In modo simile, la crittografia a chiave pubblica può essere utilizzata anche per firmare un messaggio. Alice firma il suo messaggio utilizzando la propria chiave privata e la firma può essere verificata da chiunque utilizzando la chiave pubblica di Alice.

Alice



Bob



Crittografia asimmetrica (o a chiave pubblica)

Poiché non possiamo verificare se la chiave pubblica di Bob è davvero la chiave pubblica di Bob, il sistema è vulnerabile al cosiddetto attacco man-in-the-middle (anche se nelle storie, la persona al centro è una donna, Eve).

L'attacco funziona come segue:

1. Eve si assicura che Alice creda che la chiave pubblica di Eve sia quella di Bob.
2. Alice crittografa un messaggio utilizzando la chiave pubblica di Eve.
3. Eve intercetta il messaggio e lo decrittografa utilizzando la sua chiave privata.
4. Dopo aver letto e modificato il messaggio, lo crittografa di nuovo utilizzando la vera chiave pubblica di Bob.
5. Bob riceve il messaggio e lo decrittografa utilizzando la sua chiave privata.
6. Bob è scioccato nel leggere il messaggio che crede provenga da Alice.

Una soluzione è quella di utilizzare una terza parte fidata (chiamiamolo Trent) per firmare l'autenticità della chiave. Questo è chiamato certificato. Usando la chiave pubblica di Trent, tutti possono verificare il certificato ed essere certi che ciò che viene affermato essere la chiave di Bob, è davvero la chiave di Bob.

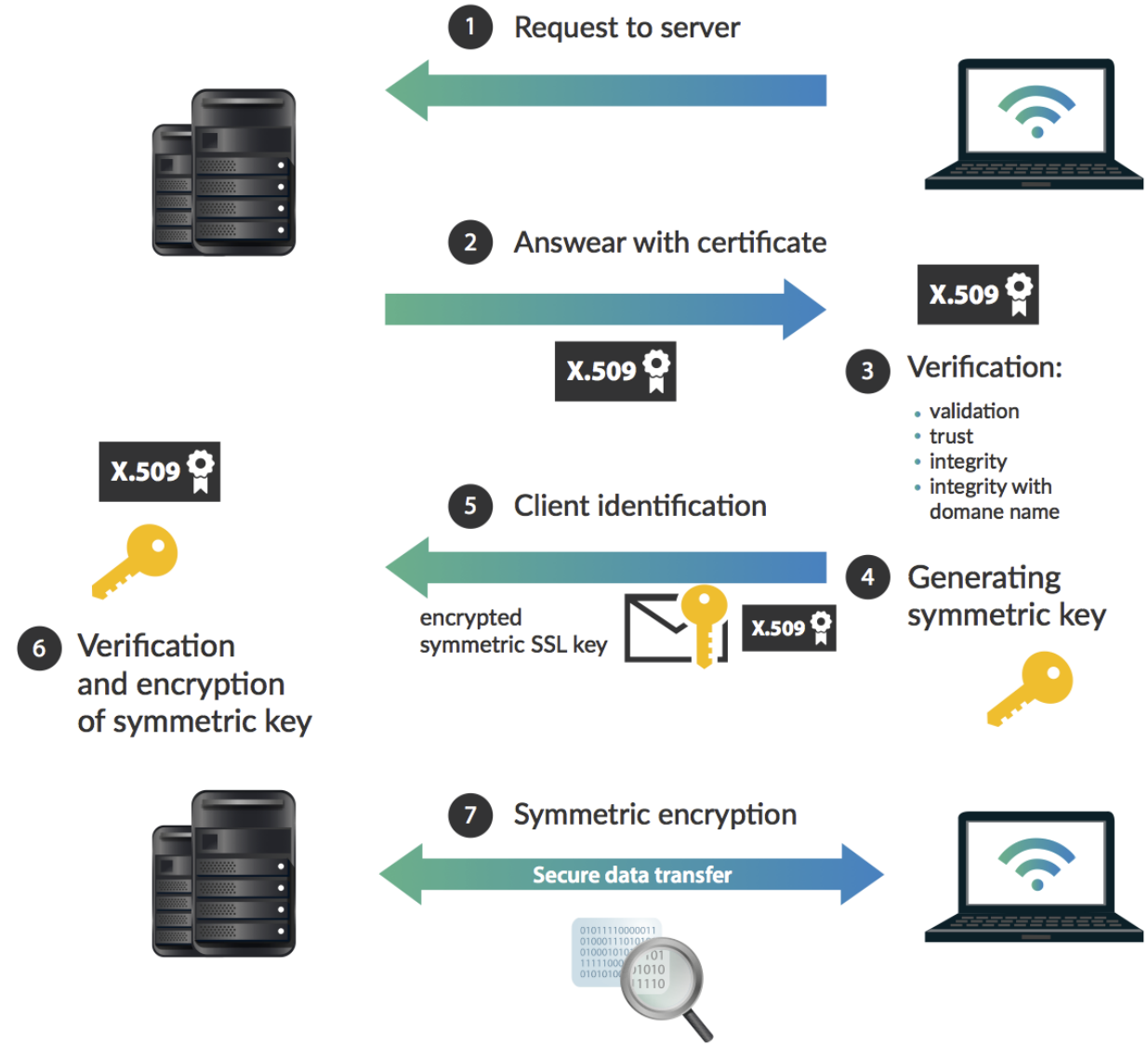
L'unico problema che rimane è che la chiave pubblica di Trent deve essere scambiata in modo sicuro. Per i certificati del sito Web, ciò viene eseguito utilizzando un cosiddetto certificato radice fornito con un browser. Poiché i certificati del sito Web e il protocollo HTTPS sono vitali per la sicurezza delle applicazioni Web, ne discuteremo più in dettaglio.

HTTPS

HTTPS è un protocollo sia per crittografare una connessione che per autenticare la parte remota.

Si basa sulla crittografia a chiave pubblica, in cui le cosiddette autorità di certificazione (CA) firmano i certificati del sito web.

L'autenticità di un'autorità di certificazione può essere stabilita perché la chiave pubblica dell'autorità di certificazione è memorizzata nel browser (nel cosiddetto certificato radice) o perché c'è un'altra CA che ha firmato un certificato per la CA (certification chain).



Ovviamente bisogna tenere in considerazione diversi aspetti nell'implementazione corretta di HTTPS

Gli errori più comuni che vengono commessi sono:

- Il web server non costringe all'utilizzo di HTTPS (e viene utilizzato HTTP)
- Gli utenti ignorano gli avvisi sui certificati non-trusted
- CA compromessi (possono rilasciare certificati per qualsiasi dominio)
- Pensare che la revoca di un certificato garantisca protezione contro attacchi MITM (non è affatto vero)
- La verifica del certificato potrebbe non essere implementata correttamente
- Encryption negotiation tra le parti comunicanti (si determina quale versione di SSL/TLS utilizzare e la cipher suite da utilizzare, operazioni incluse nel cosiddetto SSL/TLS Handshake)
- Bugs in protocolli SSL : utilizzare TLS

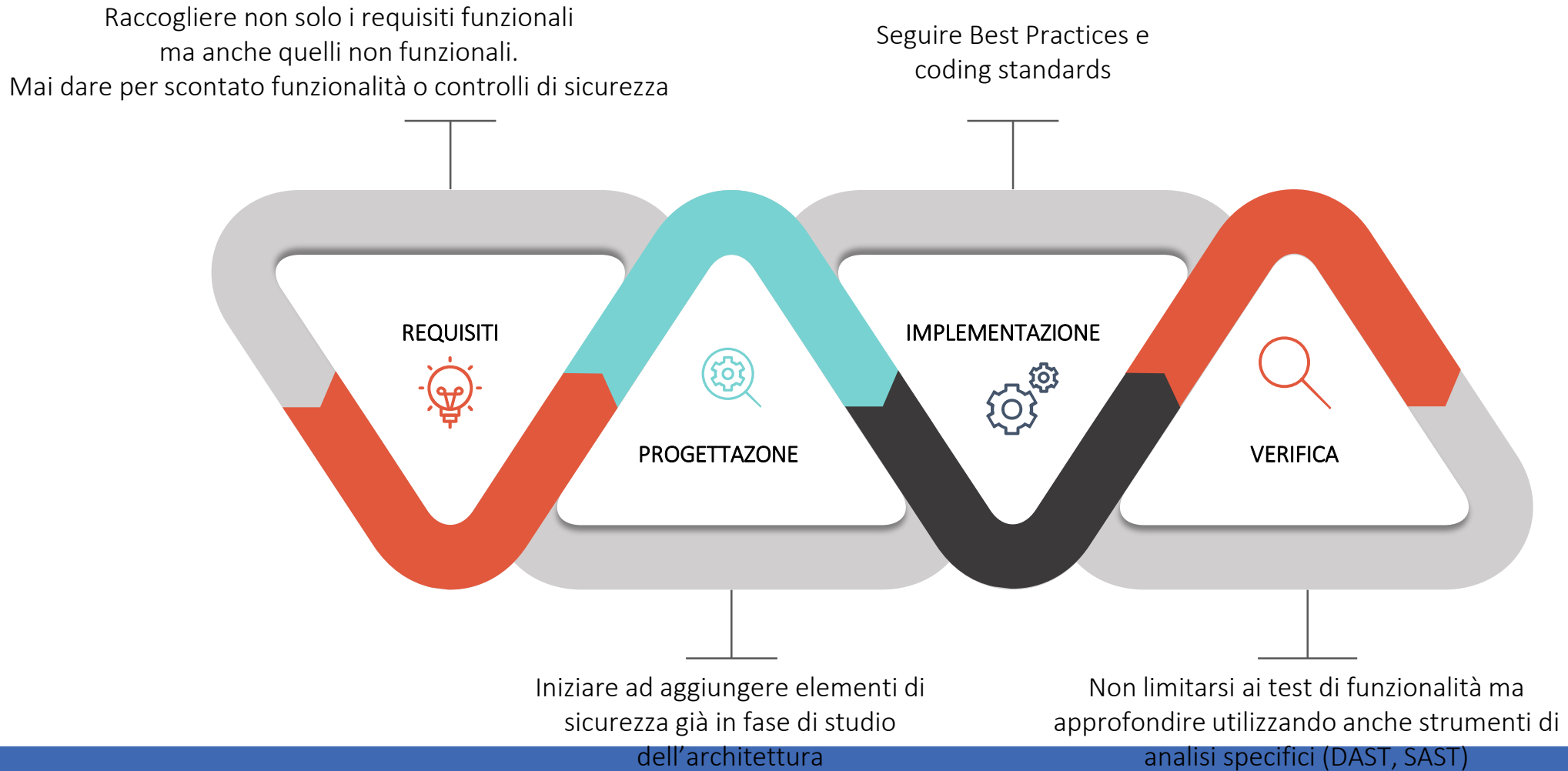
Modulo VII

Secure Software Engineering

- Secure Software Development Life Cycle
- Security Requirements
- Secure Architecture e Threat Modelling
- Coding standards e code review
- Security Testing
- Interpretazione dei risultati di test

Secure Software Engineering

Bisogna avere in mente la sicurezza in ogni fase del ciclo di vita del software



Secure Software Engineering

UTILIZZIAMO STRIDE PER OGNI FASE DEL CICLO DI VITA DEL SOFTWARE PER INDIVIDUARE PUNTI DEBOLI E CONTROMISURE

Classe di attacco	Proprietà di sicurezza interessata	Misura/contromisura
Spoofing	Autenticazione	Passwords, MFA, Certificati, Anti-CSRF
Tampering	Integrità	Corretta gestione dell'input utente
Repudiation	Non-repudiation	Logging
Information Disclosure	Confidenzialità	Crittografia, corrette cofigurazioni, gestione eccezioni
Denial of service	Disponibilità	Limitare l'utilizzo di risorse
Elevation of privilege	Autorizzazione	Configurare correttamente i permessi sulle risorse, implementare i controlli di accesso

Requisiti di sicurezza

Identificare informazioni e processi da proteggere

Chiedere di requisiti speciali per ogni “asset” dell’applicazione

I requisiti devono essere espliciti

- Nessuna ambiguità
- Nessun assunzione nascosta (non dare niente per scontato)
- Specificare sempre cosa **è permesso** e cosa **NON è permesso**

Se un requisito di sicurezza viene aggiunto ad un sistema non bisogna limitarsi a verificare la parte interessata ma l’intero sistema, poiché noi non conosciamo tutte le possibilità che un attacker può sfruttare

Spesso è difficile tradurre i requisiti non funzionali di sicurezza in controparti funzionali concrete.

Se si desidera proteggere il pannello di amministrazione da accessi non autorizzati, è necessario definire il flusso di accesso e diversi ruoli utente come comportamento del sistema o azioni dell'utente.

Se la sicurezza si basa su standard e metodi di crittografia specifici, questi standard non descrivono direttamente il comportamento di un sistema, ma piuttosto aiutano gli ingegneri con guide all'implementazione.

Requisiti di sicurezza

Come avvicinarsi

Definire minacce specifiche da cui si vuol proteggere il sistema, ad esempio:

- in quali circostanze si verifica l'accesso non autorizzato
- quali sono i precedenti della violazione dei dati
- quali tipi di attacchi malware si desidera respingere.

Espandere i requisiti non funzionali a quelli funzionali, ad esempio:

- uno schema completo di autorizzazione e autenticazione per ogni attore del sistema
- Introduzione di vincoli su chi può generare, visualizzare, duplicare, modificare o eliminare i dati.

Se il sistema deve essere conforme ad alcuni standard o regolamenti di sicurezza, la sezione non funzionale è il posto migliore per loro.

Esempio di requisito di sicurezza

Il gateway di elaborazione dei pagamenti deve essere conforme allo standard PCI DSS.

Requisiti di sicurezza

Autenticazione

- Solo gli utenti autenticati possono postare commenti
- Solo gli amministratori possono modificare le informazioni degli utenti

Integrità

- Dopo un trasferimento di moneta tra due conti, l'importo totale dei due conti deve corrispondere a quello che c'era prima della transazione

Non-repudiation

- Tutte le modifiche ad un articolo devono essere loggate: data e ora, l'utente, il numero di linee cambiate

Requisiti di sicurezza

Confidenzialità

- Informazioni sulla carta di credito possono essere utilizzate solo in fase di pagamento e solo dall'utente proprietario

Disponibilità

- L'accesso all' applicazione non deve mai essere compromesso per più di 30 minuti

Autorizzazione

- Un utente può vedere solo le informazioni dei clienti della sua organizzazione
- Un editor può creare, modificare, eliminare un articolo ma non può pubblicarlo. Inoltre non può eliminare o modificare un articolo pubblicato

Secure Architecture

Security Problems

50% IMPLEMENTAZIONE

50% REQUISITI E PROGETTAZIONE

Architectural Risk Analysis

Threat Modeling

analizzare l'architettura e verificare ogni possibile attacco applicabile

DATA FLOW DIAGRAMS
(DFD)

Agent

Process

Data store

Data flow

CAPEC (Common Attack Pattern Enumeration and Classification)



CAPEC è un catalogo pubblico di modelli di attacco insieme a uno schema completo e una tassonomia di classificazione creati per aiutare nella creazione di software sicuro.



Strutturando la definizione dei modelli di attacco come parte di una più ampia architettura integrata della conoscenza della garanzia del software che include simili sforzi di strutturazione della conoscenza come CWE, CVE, MAEC e CybOX, CAPEC supporta le esigenze di sviluppatori, tester ed educatori per creare software sicuro e aiuta a migliorare la sicurezza durante tutto il ciclo di vita dello sviluppo del software.

CAPEC - Mechanisms and Domain of Attack

Mechanism of Attack

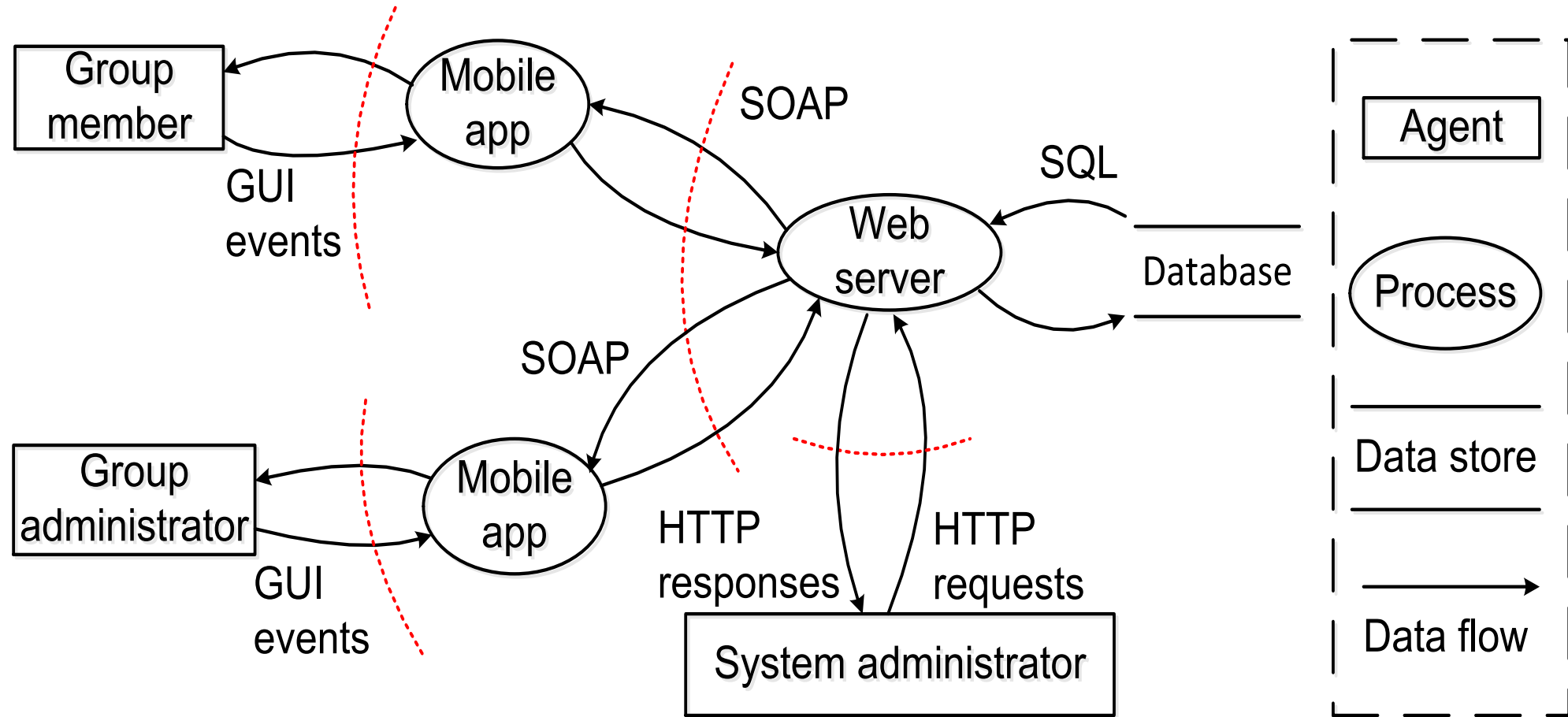
- Engage in Deceptive Interactions
- Abuse Existing Functionality
- Data Structures
- Manipulate System Resources
- Inject Unexpected Items
- Probabilistic Techniques
- Timing and State
- Collect and Analyze Information
- Subvert Access Control

Domains of Attack

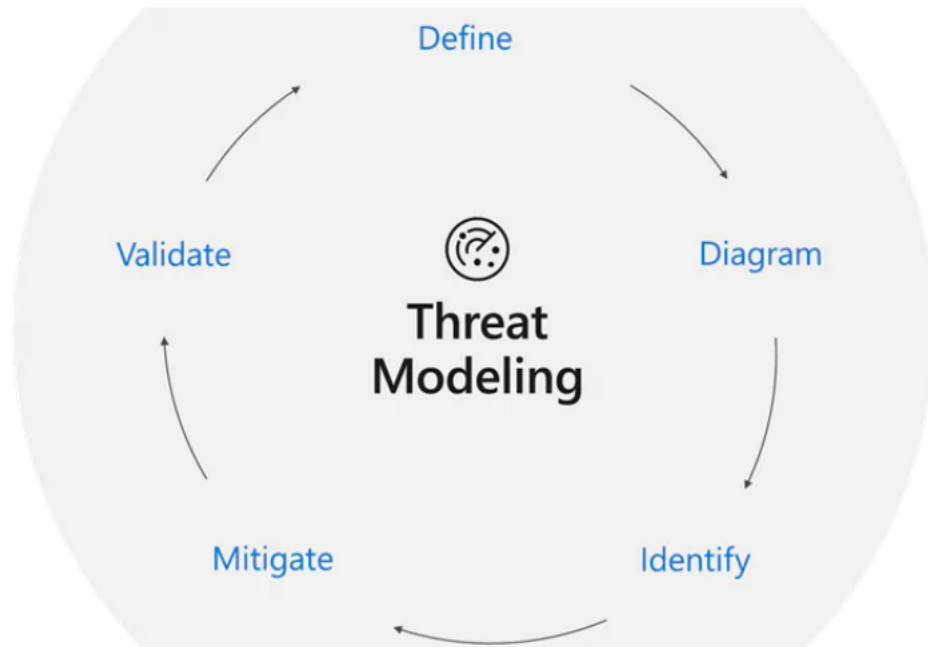
- Software
- Hardware
- Communications
- Supply Chain
- Social Engineering
- Physical Security

Secure Architecture

Data Flow Diagram



Secure Architecture



There are five major threat modeling steps:

- Defining security requirements.
- Creating an application diagram.
- Identifying threats.
- Mitigating threats.
- Validating that threats have been mitigated.

Threat modeling should be part of your routine development lifecycle, enabling you to progressively refine your threat model and further reduce risk.

Microsoft Threat Modeling Tool

The Microsoft Threat Modeling Tool makes threat modeling easier for all developers through a standard notation for visualizing system components, data flows, and security boundaries. It also helps threat modelers identify classes of threats they should consider based on the structure of their software design. We designed the tool with non-security experts in mind, making threat modeling easier for all developers by providing clear guidance on creating and analyzing threat models.

<https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling>

The Threat Modeling Tool enables any developer or software architect to:

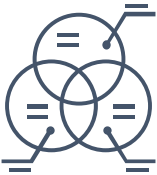
- Communicate about the security design of their systems.
- Analyze those designs for potential security issues using a proven methodology.
- Suggest and manage mitigations for security issues.

Secure Architecture - Mitigazioni



Mitigazioni tecniche

Quando un problema è stato trovato, possiamo pensare alla mitigazione. Spesso si tratterà di una mitigazione tecnica, come l'utilizzo di un determinato componente o servizio, o la programmazione in un certo modo.



Mitigazioni non-tecniche

E' anche possibile mitigare in modo non tecnico, ad esempio rilevando il problema e disponendo di uno scenario alternativo per ridurre l'impatto.



Un esempio

Un buon esempio è la mitigazione contro la creazione di copie digitali non autorizzate. Questo problema non può essere risolto facilmente (o addirittura del tutto) con la tecnologia, ma essendo in grado di rilevare copie non autorizzate e risalire al proprietario originale, abbiamo i mezzi per intraprendere azioni legali contro chi ha fatto la copia.



Cambiamenti e modifiche

La mitigazione può portare a cambiamenti nei requisiti e nell'architettura, quindi è una buona idea esaminare l'impatto sulla sicurezza dell'implementazione della mitigazione prima di decidere di implementarla.



Un esempio

Ad esempio, l'implementazione di un servizio Single Sign-On come misura di autenticazione per proteggere dagli attacchi di spoofing può essere una buona idea, ma attenzione, il sistema Single Sign-On stesso può ora diventare un obiettivo per gli attacchi, portando a ulteriori minacce.

Secure Coding



Nella fase di codifica, dobbiamo scegliere i costrutti di programmazione corretti per rendere il codice sicuro.



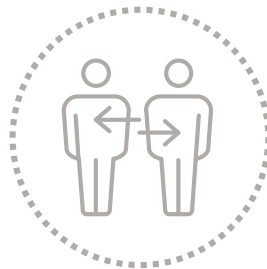
Il progetto CWE (Common Weakness Enumeration) di MITRE elenca oltre 900 punti deboli che i programmatori possono fare, alcuni dei quali sono molto oscuri.



Molti costrutti dipendono dal linguaggio di programmazione e dal framework che è stato scelto, ma la maggior parte di essi segue principi generali.



Quando si scrive codice, è importante sapere quali costrutti si dovrebbero usare e quali evitare. Di solito questo è specificato in cosiddetto Coding Standard.

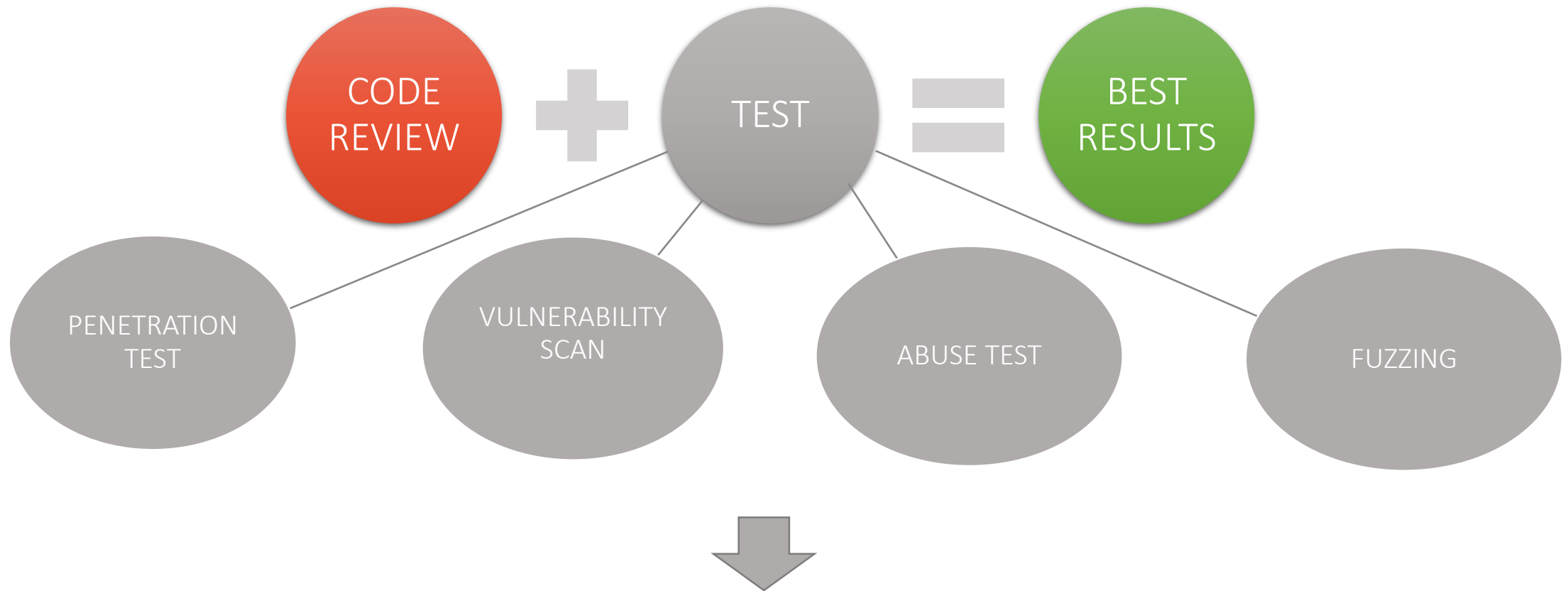


Tuttavia, fidarsi del fatto che i membri del team scelgano costrutti di programmazione sicuri è positivo, ma una qualche forma di verifica è migliore. Pertanto, le revisioni del codice (code review) sono un buon modo per rilevare gli inevitabili errori di codifica



Esistono molti strumenti automatici per risolvere questo problema. Nonostante il tempo che deve essere investito, le revisioni del codice si sono rivelate molto efficaci. Un'altra pratica che può aiutare è quella di Pair Programming

Test di sicurezza



Findings: results of tests and verifications

Penetration Test

Esperti in sicurezza attaccano il nostro sistema prima che possa farlo un attacker



Utilizzati in fase di sviluppo sensibilizzano il team ad adottare pratiche di sviluppo di software sicuro



Di solito possono essere effettuati in una fase già avanzata dello sviluppo in quanto necessitano di un sistema funzionante

Vulnerability Scan

Strumenti per la verifica automatica delle vulnerabilità di un sistema, da quelle relative all'hardening agli attacchi XSS



Possono rilevare vulnerabilità a noi sconosciute, ma non agli attacker (utilizzano gli stessi tools)



Possono rilevare vulnerabilità NOTE

solo

Abuse Test

Test implementati, di norma, prima dello sviluppo del software. Possono essere sviluppati dal team stesso, in quanto a conoscenza di specifiche vulnerabilità non rilevabili da altre tipologie di test



Richiedono minimo aiuto da parte di esperti in sicurezza



Se sviluppati da team con poca competenza a livello di sicurezza possono risultare inutili

Fuzzing

Si studia il comportamento del sistema quando sottoposto ad input generato o modificato in modo casuale. Di norma utilizzati in scenari di elaborazione complessi per cui non è possibile definire test specifici



Utili per il rilevamento di vulnerabilità e bug.

Se correttamente impostati possono essere eseguiti per lungo tempo portando a risultati sorprendenti



Bisogna eseguire determinate procedure di setup e monitoraggio, se non progettati bene non garantiscono risultati soddisfacenti

Feedback dei test



Se si utilizzano i risultati di queste verifiche (chiamati *findings*) per migliorare sia il processo di sviluppo che le verifiche, queste reti di salvaguardia diventeranno più forti ed efficaci nel tempo.



Un finding può non solo portare a modifiche nel codice, ma anche nei requisiti e nell'architettura del sistema. Tali modifiche possono influire sulla sicurezza e richiedere una nuova analisi della sicurezza.



È importante osservare questo ciclo di feedback. I risultati possono anche portare a migliori reti di salvaguardia.



Ad esempio, i risultati di un penetration test possono portare a test di abuso aggiuntivi, oppure i risultati di una revisione del codice possono portare a uno standard di codifica sicuro rivisto e strumenti automatizzati per rilevare questi costrutti di programmazione non sicuri.



L'implementazione di queste pratiche può rappresentare una sfida, ma alla fine porterà benefici.

- OWASP Cheat Sheets

- <https://cheatsheetseries.owasp.org/>

- Microsoft SDL

- <https://www.microsoft.com/en-us/securityengineering/sdl/>

- Framework Secure Software

- <https://securesoftwarealliance.org/framework-secure-software/>